

大语言模型的发展推动人工智能应用从问答交互走向具备任务规划、工具调用和流程执行能力的 AI Agent。企业业务场景在降低应用构建门槛的同时，也要求执行过程可观察、关键结果可校验、操作记录可追溯。现有低代码编排平台和 Agent 开发框架能够支持流程搭建与自动执行，但在 Java 业务系统中的可视化编排、图结构调度和人工干预实践仍相对有限，许多工作流启动后连续执行至结束，缺少在关键节点暂停、审核、修改中间结果并恢复执行的机制。针对上述问题，本文设计并实现一套基于 Spring Boot 的 AI Agent 智能体编排系统。系统采用前后端分离与分层架构，前端基于 React、TypeScript 和 React Flow 构建可视化流程画布，后端按接口层、应用层、领域层和基础设施层组织核心逻辑；通过版本快照保存 Agent 配置，基于 DAG 调度推进节点状态，并利用 SSE 推送运行进度；同时在执行前后引入人工检查点，并通过审批记录持久化保证过程追溯。系统还集成知识文档存储与向量检索能力，并通过 Docker Compose 完成容器化部署验证。运行结果表明，系统实现了从 Agent 创建、流程发布、自动执行、人工审核到流程恢复的核心功能闭环，对 Java 生态下构建可控、可追踪的智能体执行平台具有一定工程实践价值。

The development of large language models is driving artificial intelligence applications from question-answering interaction toward AI Agents capable of task planning, tool invocation and workflow execution. In enterprise scenarios, agent applications need not only low development barriers, but also observable execution, verifiable key results and traceable operations. Existing low-code orchestration platforms and agent development frameworks can support workflow construction and automated execution, but visual orchestration practices in Java-based business systems are still relatively limited. In addition, many workflows run continuously from start to finish once triggered, lacking mechanisms to pause at key nodes, conduct human review, modify intermediate results and resume execution. This makes it difficult to meet the requirements of high-risk business processes that combine automation with human control.

To address these issues, this thesis designs and implements an AI Agent orchestration system based on Spring Boot. The system targets application builders and business users, aiming to provide integrated capabilities for visual modeling, controllable execution, human review and execution tracking. It adopts a separated front-end/back-end architecture and a layered back-end design, organizing business logic into the interface, application, domain and infrastructure layers. The front end is built with React, TypeScript and React Flow to provide a visual workflow canvas. The core modules include agent management, workflow scheduling and execution, human review with workflow resumption, knowledge-base retrieval enhancement, and user authentication with security support.

In implementation, the system stores agent configurations through version snapshots, advances workflow node states through DAG scheduling, and pushes real-time execution progress to the front end through SSE. To address the lack of human control at key nodes, the workflow engine introduces two types of human checkpoints: before node execution and after node execution. When a workflow reaches a checkpoint, execution is paused and a pending review task is created. Reviewers can inspect the context, approve the workflow with modified intermediate results, or reject and terminate the workflow. A Redis-based queue, distributed locks, optimistic locking and persistent review records are used to ensure state consistency and process traceability under concurrent review scenarios. The system also supports knowledge enhancement and secure access through asynchronous document processing, Milvus vector retrieval and user authentication. The running results show that the system implements the core functional closed loop from agent creation, workflow publishing and automated execution to human review and workflow resumption, demonstrating practical engineering value for building controllable and traceable agent execution platforms in Java-based business systems.

绪论

选题背景及意义

随着大语言模型（Large Language Model, LLM）推理能力、工具调用能力和上下文理解能力的快速发展，人工智能应用正从传统的问答式交互逐步扩展到能够理解任务目标、规划执行步骤并调用外部工具完成复杂任务的智能体（AI Agent）形态。与单轮问答系统相比，AI Agent 更强调任务拆解、流程控制、状态记忆和外部环境交互，能够在一定程度上把大模型能力嵌入到真实业务流程之中。围绕 AI Agent 的开发、编排和部署，业界已经出现了 Dify、Coze 等拖拉拽式低代码平台，也出现了 LangGraph、AutoGen 等面向开发者的编排框架，这些工具降低了智能体应用从原型验证走向工程实践的门槛。

然而，从企业级后端系统建设的角度看，现有 Agent 编排能力仍存在两方面不足。首先，较成熟的 Agent 编排工具和开发生态更多集中在 Python、JavaScript 或独立低代码平台中，而许多企业核心业务系统仍长期采用 Java 和 Spring Boot 技术体系构建。Java 生态并非缺少大模型接入能力，相关框架已经提供了模型调用、提示词模板、结构化输出和函数调用等基础能力，但面向业务流程的可视化 Agent 编排、图结构运行和状态恢复实践相对较少，开发者在将 Agent 工作流嵌入既有 Java 后端系统时仍需要进行较多工程化设计。

其次，现有拖拉拽式低代码平台通常以“配置完成后自动执行”为主要范式， workflow 一旦启动便从起点连续运行到终点，更适合自动化程度较高、风险较低的任务场景。在涉及内容审核、业务审批、执行结果确认或关键决策的场景中，系统需要在特定节点暂停执行，允许人工查看上下文、校验中间结果、进行必要修改，并在审批通过后恢复流程或在审批拒绝后终止流程。若缺少这种人工审核与流程恢复机制，智能体虽然能够自动执行任务，却难以满足企业场景对过程可控、结果可校验和操作可追溯的要求。

针对上述问题，本课题设计并实现一套基于 Spring Boot 的 AI Agent 智能体编排系统。系统以 Java 后端工程实践为基础，将可视化 workflow 编排、DAG 调度执行、人工审核检查点、知识库检索增强和用户认证安全能力结合起来，使应用构建者能够通过前端画布配置 Agent workflow，使业务使用者能够触发已发布流程，并在关键节点通过人工审核参与执行过程。该课题的研究意义在于：一方面探索 Java 技术栈下智能体可视化编排系统的实现路径，另一方面将人工审核机制嵌入 Agent 执行链路，为需要“自动执行 + 人工把关”的业务场景提供一种可参考的工程方案。

国内外研究现状

近年来，基于大语言模型的自主智能体已成为人工智能领域的重要研究方向。Xi 等系统梳理了 LLM Agent 的概念框架、能力构成与应用边界，Wang 等则从智能体构建、任务执行、评估方法与应用场景等维度总结了大语言模型驱动自主智能体的发展脉络。围绕智能体的可视化编排与工程化实现，现有研究与产品实践主要集中在 Agent 编排平台与开发框架、Java 生态中的大模型应用支持、人工介入机制以及面向外部知识接入的检索增强生成等方向。

AI Agent 编排平台与开发框架现状

面向应用快速搭建的低代码平台发展迅速。Dify、Coze 等产品提供了可视化的 Agent 搭建界面；国内方面，百度、阿里、腾讯等厂商也推出了面向应用开发者的低代码构建能力。相关研究已经开始将 Dify workflow 能力用于测试项批量生成等场景。这类平台的优势在于入门门槛低、节点配置直观、适合快速原型验证，但其运行时通常由平台自身托管，业务系统若要复用既有的数据模型、权限体系、审计规则和事务边界，往往仍需要额外的二次开发与适配。

在编程框架层面，Python 生态较为活跃。LangChain 是目前常用的 LLM 应用开发框架，提供丰富的模型接口、工具集成与记忆管理能力，但早期链式调用方式在表达复杂分支、循环和状态恢复时存在一定局限。为解决这一问题，LangGraph 引入图结构建模思想，将 Agent 的状态流转抽象为节点与边的组合，支持循环、分支、持久化执行和人工介入，成为当前智能体 workflow 编排的重要方案。微软研究院推出的 AutoGen 则采用“对话即计算”范式，让多个 Agent 以对话方式完成任务分解与协作。在方法论层面，ReAct 将

“推理”与“行动”交织为统一决策范式，为工具调用型 Agent 提供了重要基础。上述框架为智能体编排提供了成熟参考，但对以 Java 和 Spring Boot 为主的企业后端系统而言，仍存在语言栈差异、运行环境隔离和业务系统集成等问题。

Java 生态与人工审核现状

在 Java 生态侧，Spring 体系在企业级应用开发中已经形成较成熟的工程实践，能够为大模型应用提供稳定的 Web 接口、依赖管理、事务处理和安全控制基础。但从系统编排角度看，现有 Java AI 应用框架更偏向 LLM 应用基础设施层，主要解决模型调用和外部工具接入问题，并不直接提供完整的可视化流程编排、图结构调度、执行状态持久化和人工审核恢复能力。Java 生态中与 LangGraph 类似的、面向 Agent 工作流的图导向编排实现相对较少，开发者若要在 Spring Boot 系统中实现节点调度、上下文传递、执行暂停和恢复，仍需要结合具体业务进行较多设计。

人工审核是企业业务流程中常见的控制机制，例如内容发布前审核、关键操作二次确认、自动生成结果复核和异常流程人工处理等。传统业务系统通常会将审批记录、操作人、审批意见和状态流转持久化，以便满足审计和追溯要求。相比之下，许多拖拉拽式 Agent 或 workflow 平台更重视自动化执行效率，缺少将“暂停—审核—修改—恢复”作为一等能力纳入运行时的设计。即使部分框架提供了人工介入入口，在与具体业务用户、权限控制、审核记录、并发审批和流程恢复语义结合时，也仍需要进一步工程化实现。

因此，在 Java 技术栈下设计一套既支持可视化编排，又能在执行过程中引入人工审核检查点的 Agent 编排系统，具有较强的实践意义。该类系统不应只关注模型调用结果，还需要管理工作流图结构、节点状态、执行上下文、审核队列和恢复策略，从而使智能体执行过程具备更好的可控性和可追溯性。

知识检索增强与向量数据库现状

在大模型应用中，检索增强生成（Retrieval-Augmented Generation, RAG）是缓解模型“幻觉”、引入外部知识的常见技术路径。Lewis 等最早提出了 RAG 范式，将参数化模型与外部检索器结合，使生成结果具备可追溯的知识依据。国内研究也对基于大型语言模型的检索增强生成技术进行了综述，梳理了 RAG 的基本流程、评估基准和未来挑战。落到工程实现，向量数据库 Milvus 提供了高维向量管理与近似最近邻检索能力，是支撑 RAG 应用的常用基础设施之一。对于需要接入业务文档或领域资料的 Agent 应用而言，在工作流执行过程中引入知识检索节点，有助于为 LLM 节点提供更具针对性的上下文信息。

综合以上分析，现有研究与产品已经在智能体框架、低代码编排和知识增强方面取得了较多进展，但在同时兼顾 Java 技术栈集成、可视化 workflow 编排、节点级人工审核、流程恢复和知识检索能力方面仍存在一定不足。因此，本文围绕上述问题设计并实现基于

Spring Boot 的 AI Agent 智能体编排系统，以验证相关能力组合在原型系统中的可行性。

研究目标与内容

本课题的研究目标是在 Spring Boot 技术体系下设计并实现一套 AI Agent 智能体编排系统，使系统能够支持智能体工作流的可视化配置、后端调度执行、人工审核干预、知识检索增强与基础安全认证。围绕上述目标，本文主要工作如下：

- (1) 调研国内外低代码 Agent 平台、Python 编排框架、Java 生态大模型应用和人工介入机制相关研究，分析智能体编排、工作流执行和人工审核机制结合方面的现状与不足，为系统的功能定位与技术选型提供依据。
- (2) 面向应用构建者和业务使用者等参与角色，分析 Agent 创建发布、工作流运行、人工审核、知识库管理和用户认证等典型业务流程，构建系统的功能性需求与非功能性需求模型。
- (3) 设计并实现基于 Spring Boot 的 AI Agent 智能体编排系统，覆盖 Agent 可视化编排、工作流执行与调度、人工审核检查点与流程恢复、知识文档存储与智能检索、用户认证与系统安全五项核心能力。
- (4) 围绕工作流运行时设计 DAG 调度、执行上下文传递、节点状态推进和 SSE 实时推送机制，并在关键节点引入人工检查点，使系统能够支持执行暂停、审批通过恢复、审批拒绝终止和审批记录追溯。
- (5) 面向核心功能与典型业务链路开展单元测试、功能测试、性能验证和部署验证，检查系统五项核心能力的功能闭环、关键链路可用性、基本性能表现和容器化部署可行性。

论文组织结构

本文围绕基于 Spring Boot 的 AI Agent 智能体编排系统展开，按照“背景分析—需求建模—概要设计—详细实现—测试部署—总结展望”的逻辑组织。

第 1 章为绪论，介绍课题背景与研究意义，梳理国内外研究现状，并明确本文的研究目标、主要工作和组织结构。

第 2 章为智能体编排系统需求分析，从用户角色、业务流程、功能需求和非功能需求等方面明确系统建设目标。

第 3 章为智能体编排系统概要设计，说明系统总体架构、功能模块划分、数据模型、核心状态流转和关键流程设计。

第 4 章为智能体编排系统详细设计与实现，重点阐述用户认证、Agent 管理、工作流调度、人工审核、知识库检索和前端界面等核心模块的实现过程。

第 5 章为智能体编排系统测试与部署，从单元测试、功能测试、性能表现和容器化部署等方面验证系统可用性与部署可行性。

最后为智能体编排系统总结与展望，总结全文工作，分析当前系统的不足，并对后续可扩展方向进行展望。

智能体编排系统需求分析

业务需求分析

业务背景

随着大语言模型能力逐步增强，企业开始尝试将大模型嵌入客服问答、资料检索、报告生成、流程审批和业务辅助决策等场景。成熟的低代码智能体平台以 Dify 等工具为代表，通常围绕应用构建、可视化工作流、知识库检索增强、工具调用、应用发布与运行观测等能力展开，帮助用户将大模型能力快速组合成可运行的业务应用。

然而，在企业内部系统落地过程中，仅具备“创建一个对话机器人”并不足以支撑严肃业务场景。一方面，业务流程通常不是单轮问答，而是由多个步骤组成：需要接收输入、查询知识、调用模型、判断条件、生成结果并记录执行过程；另一方面，部分任务涉及合同内容、客户回复、审核意见或内部决策，不能完全交由模型自动完成，需要在关键节点保留人工检查与修正入口。因此，本系统的业务目标是提供一套面向企业业务流程的 AI Agent 编排平台，使智能体应用能够被构建、发布、执行、审核、追踪和维护。

核心业务需求

围绕上述业务背景，本系统需要满足以下五类核心业务需求。

(1) 智能体应用快速构建需求。企业需要将具体业务流程抽象为可执行的智能体应用。系统应支持通过可视化方式创建 Agent、配置节点、连接流程、保存草稿并发布版本，使应用构建者能够在较低编码成本下完成业务流程编排。同时，业务流程会持续迭代，系统需要支持版本发布、历史回滚与版本追溯，避免流程调整后无法恢复到稳定版本。

(2) 业务任务可触发、可观察的执行需求。业务使用者需要像使用普通业务系统一样触发已发布的智能体应用，并在执行过程中看到节点状态、模型输出和最终结果。由于大模型调用和知识检索存在耗时，系统需要提供实时反馈能力，避免用户只能在长时间等

待后得到一个不可解释的最终答案。因此， workflow 执行过程必须具备状态推进、实时推送、执行日志与异常反馈能力。

(3) 关键节点人工审核需求。在企业业务场景中，模型生成结果可能影响后续决策或外部输出。当执行链路进入高风险节点时，系统应允许业务使用者对节点输入或输出进行检查，必要时修改中间内容，再决定通过或拒绝。该需求要求系统能够在运行中暂停 workflow、保存上下文、等待人工处理，并在审批通过后恢复执行或在审批拒绝后终止执行。

(4) 业务知识支撑需求。企业应用不能只依赖大模型的通用知识，还需要接入产品文档、业务制度、FAQ、操作手册和历史资料等内部知识。系统应支持知识数据集管理、文档上传、异步解析、文本分块、向量化入库与语义检索，并在 workflow 执行过程中将检索结果注入模型上下文，从而提高回答的准确性、时效性和可追溯性。

(5) 账号安全与访问控制需求。智能体应用涉及账号身份、业务数据、审批记录和知识文档，因此需要基础的账号注册、登录认证、访问控制、密码保护和操作留痕能力。系统应对未认证请求进行拦截，对验证码发送、密码强度、登录失败次数等环节进行限制，并对关键业务操作进行记录，确保平台在演示和业务验证阶段具备基本的安全性与可追溯性。

业务参与人员

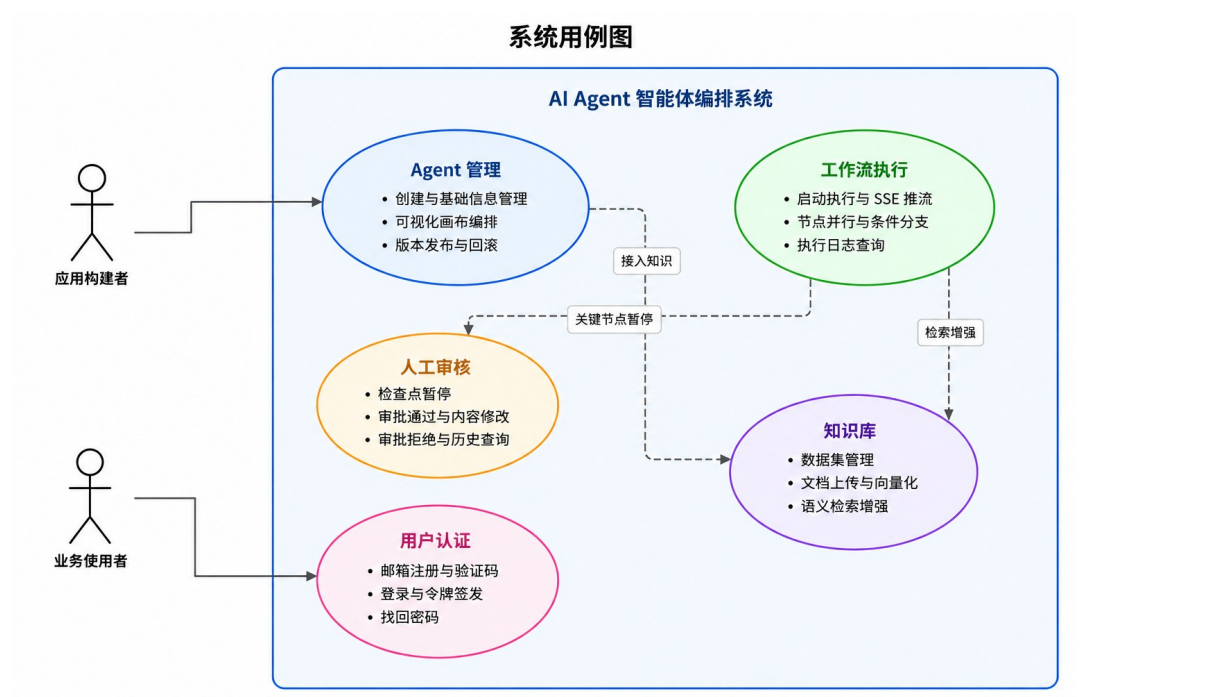
根据上述业务需求，本系统的参与人员可归纳为两类：应用构建者和业务使用者。这两类人员不是从具体技术岗位直接划分而来，而是由智能体应用从构建、发布到运行、审核的业务闭环推导得到。

(1) 应用构建者。应用构建者负责将业务流程转化为可执行的 Agent workflow。其主要关注 Agent 的创建、画布编排、节点参数配置、知识节点接入、版本发布和历史回滚。该角色既可以是具备技术背景的开发人员，也可以是熟悉业务流程并具备平台配置能力的业务实施人员。

(2) 业务使用者。业务使用者负责调用已经发布的智能体应用完成具体任务。其主要关注任务输入、执行进度、模型输出、最终结果、审批决策和历史记录。当 workflow 进入人工检查点时，业务使用者还需要根据业务规则对中间结果进行确认、修正、通过或拒绝。

系统功能需求分析

基于业务需求与参与人员分析，本系统需要围绕 Agent 管理、workflow 调度执行、人工审核、知识库检索增强和用户认证五个方面形成完整功能闭环。系统用例图如图 2.1 所示。



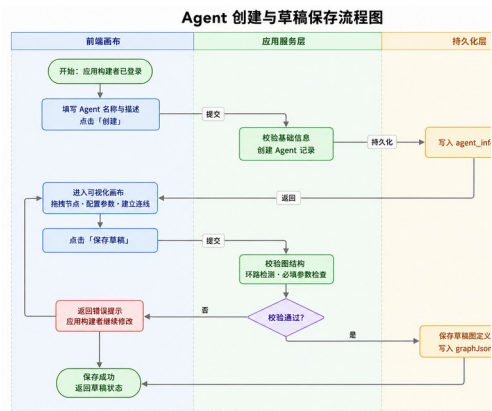
系统用例图

Agent 管理与可视化编排需求

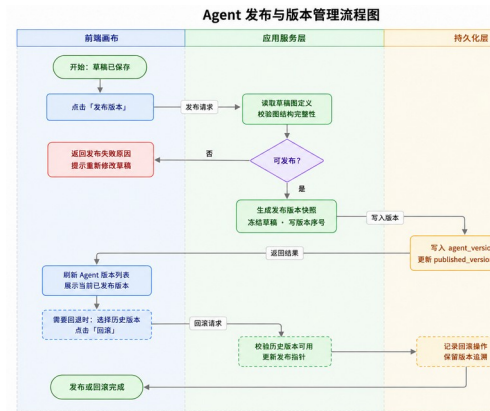
Agent 管理是系统的入口能力。应用构建者首先需要创建 Agent，录入名称、描述等基础信息，随后进入可视化画布完成节点编排。画布应支持配置 LLM 节点、条件节点、知识检索节点、工具节点等不同类型的执行单元，并通过连线描述节点之间的依赖关系。

系统在保存草稿和发布版本时需要进行图结构校验。如果图中存在环路、孤立节点或必要参数缺失，系统应拒绝保存或发布并给出明确错误提示。发布后的 Agent 版本应作为可执行快照保存，保证后续工作流执行时不会受到草稿继续编辑的影响。当业务流程调整出现问题时，应用构建者应能够查看历史版本并回滚到稳定版本。

Agent 创建、草稿保存、发布与版本管理流程如图 2.2 和图 2.3 所示。



创建与草稿保存



发布与版本管理

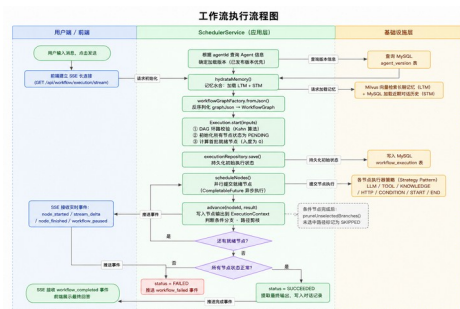
Agent 创建与发布流程图

工作流调度执行需求

工作流执行是业务使用者直接感知系统价值的核心链路。业务使用者选择已发布的 Agent 后输入任务内容，系统应加载对应版本的图定义、初始化执行上下文并启动调度。在执行过程中，系统需要按照节点依赖关系推进状态，对无依赖冲突的就绪节点支持并行执行，对条件分支节点支持路径选择和无效路径剪枝。

由于大模型调用和知识检索具有异步、耗时和结果不确定等特点，系统需要通过服务端推送事件实时返回节点启动、节点完成、节点失败、大模型流式输出和最终完成等事件。业务使用者应能够在前端界面实时看到执行状态，并在执行失败时根据错误信息定位问题。系统还应持久化执行记录和关键节点日志，为后续复盘、审计和优化提供依据。

工作流执行的完整流程如图 2.5 所示。



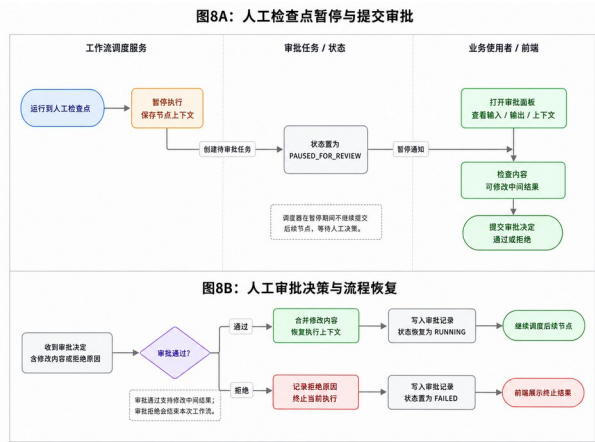
工作流执行流程图

人工审核与流程恢复需求

人工审核是本系统区别于普通自动化 Agent 平台的重要能力。应用构建者可以在关键节点配置人工检查点，检查点可设置在节点执行前或执行后。当 workflow 运行到检查点时，系统应暂停当前执行，保存执行上下文和待审核内容，并向前端推送暂停事件。

业务使用者在审批界面中查看当前节点的输入、输出和上下文信息，并根据业务要求选择直接通过、修改中间结果后通过，或拒绝并终止 workflow。系统在审批提交时需要校验执行状态和版本号，防止多人同时审批造成状态覆盖，同时记录审批人、审批时间、审批结果、修改内容和备注，以满足业务追溯要求。

人工检查点审批流程如图 2.6 所示。



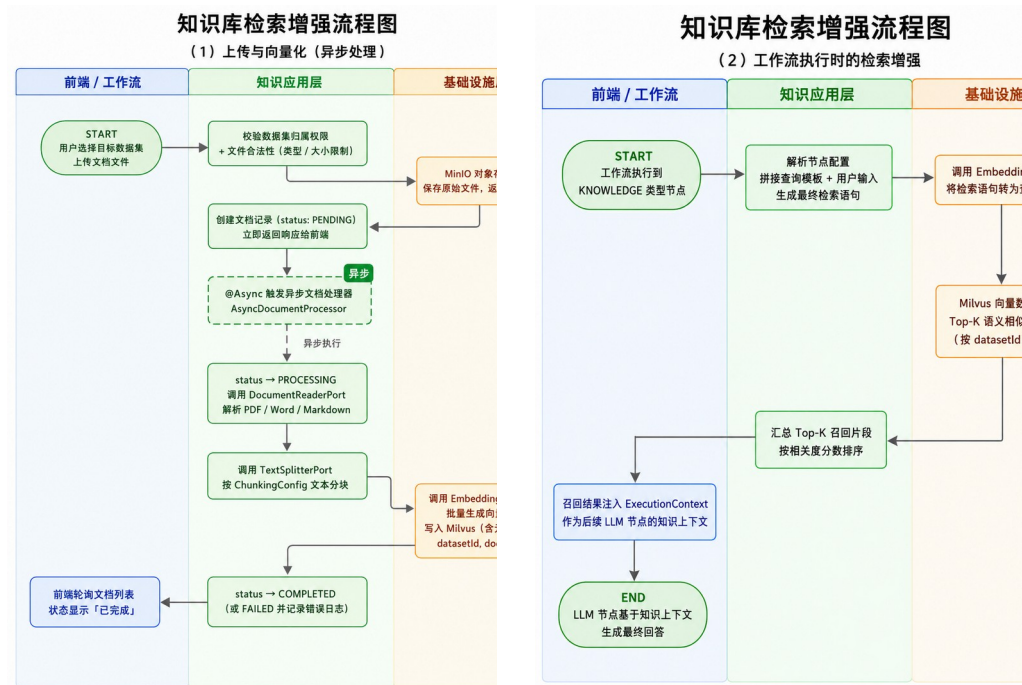
人工检查点审批流程图

知识库与检索增强需求

知识库能力用于为智能体应用提供企业内部知识支撑。应用构建者可以创建知识数据集并上传业务文档。系统需要对上传文件进行类型、大小和内容合法性校验，随后将原始文件保存至对象存储，并异步完成文档解析、文本分块、向量生成和向量入库。文档处理完成后，系统应记录处理状态，失败时给出可追踪的错误信息。

在工作流运行时，知识检索节点根据用户输入或节点模板生成检索语句，调用向量检索服务召回相关片段，并将检索结果注入后续大模型节点上下文。该能力能够减少模型幻觉，使模型输出更加贴合企业资料和业务规则。当向量检索服务不可用时，系统应记录异常并支持降级执行，避免知识检索失败直接阻断非关键业务链路。

知识库从文档上传入库到运行时检索增强的流程如图 2.9 所示。



上传与向量化 (异步处理)

workflow 执行时的检索增强

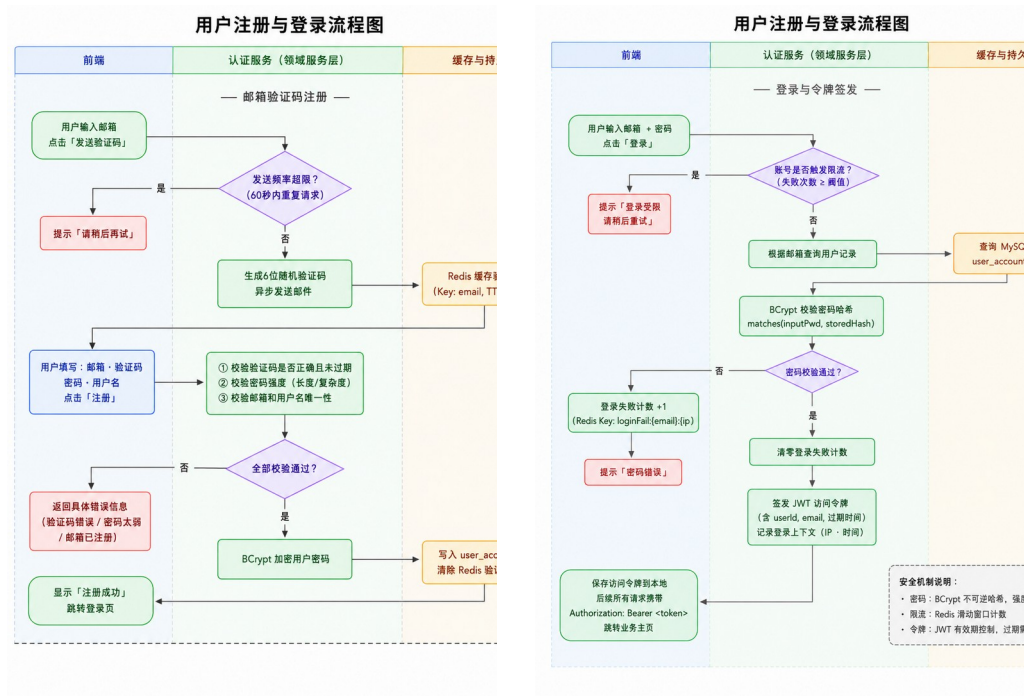
知识库检索增强流程图

用户认证与安全支撑需求

用户认证是系统所有业务功能的访问基础。应用构建者和业务使用者均需要先完成注册和登录，在获得访问令牌后才能使用对应功能。系统应支持邮箱验证码注册、账号登录、找回密码和令牌签发，并对验证码发送频率、密码强度、登录失败次数和重复注册进行限制。

认证成功后，前端在后续请求中携带访问令牌，后端根据令牌识别用户身份并进行访问控制。对于 Agent 管理、工作流执行、审批决策、知识库维护等业务接口，系统应拒绝未认证请求。

用户注册与登录流程如图 2.12 所示。



邮箱验证码注册

登录与令牌签发

用户注册与登录流程图

非功能需求

性能需求

系统应在企业级常规并发量下保持稳定响应。Agent 管理、用户认证、知识库查询等管理类接口应具备良好的实时交互体验；工作流执行过程中的节点状态推送与模型生成结果回显应及时、连贯，避免出现明显卡顿或长时间无反馈。知识文档上传应快速返回接收结果，解析与向量化处理在后台异步完成，避免大文件处理阻塞用户界面。

可靠性需求

工作流执行状态、节点执行记录、审批记录、Agent 版本和文档处理状态均须持久化保存。系统重启后，历史数据应完整可查，失败任务应保留错误信息和上下文。在多人并发审批或重复提交场景下，系统需要通过版本号校验或锁机制保证状态一致性，防止同一待审批任务被重复处理。

可扩展性需求

系统应支持后续扩展新的节点类型、模型厂商、工具适配器、向量数据库和对象存储实现。节点执行逻辑与底层资源访问应通过接口隔离，避免新增节点或替换基础设施时影响已

有业务主链路。工作流图结构、节点参数和执行上下文也应保持相对通用，为后续接入更多业务场景提供扩展空间。

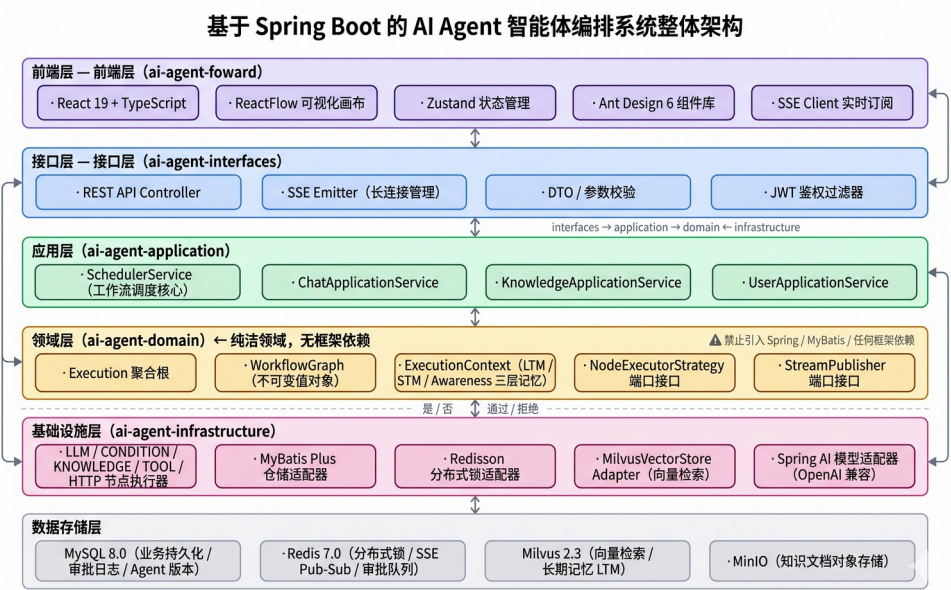
安全性需求

所有业务接口均应基于身份认证后访问，未认证请求需被拒绝。用户密码不得明文存储，应采用安全哈希算法保存。邮箱验证码、登录入口和找回密码入口应具备频率控制能力，防止恶意刷取验证码和暴力破解。审批操作、关键执行状态和知识文档处理记录应完整留痕，满足业务审计和问题追溯要求。

智能体编排系统概要设计

系统结构设计

系统采用分层架构与前后端分离的部署模式，后端按照接口层、应用层、领域层、基础设施层四层划分，前端基于 React 与 TypeScript 实现可视化画布与业务交互界面。系统整体结构如图 3.1 所示。



系统整体结构图

各层的核心职责如下：

接口层：对外提供 RESTful 接口与实时推送通道，负责处理 HTTP 请求、参数校验以及数据传输对象与领域对象之间的转换。用户注册、登录、知识库管理、审批接口与工作流启动接口均在此层对外暴露。

应用层：包含 Agent 管理服务、 workflow 调度服务、知识应用服务与用户应用服务等。该层负责跨聚合的编排，例如协调 Agent 查询、 workflow 图结构解析、知识检索增强、审批恢复与异步文档处理等，但不承载具体的领域规则本身。

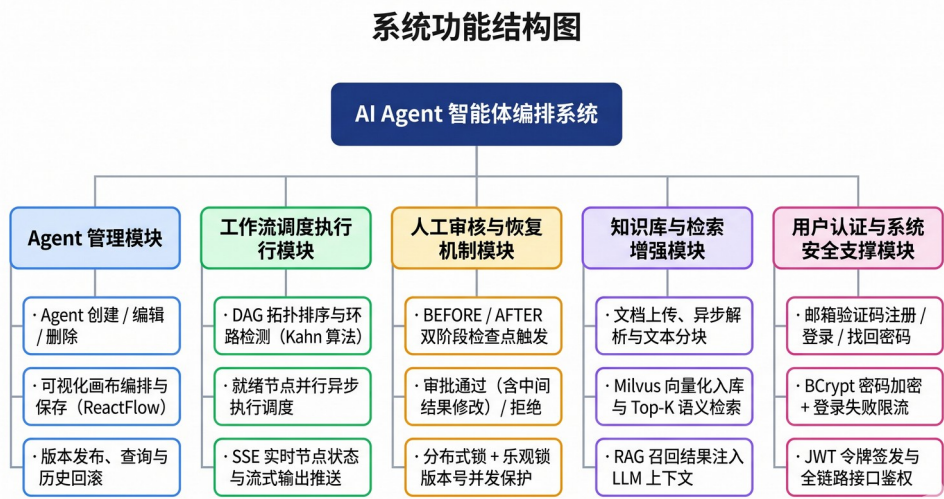
领域层：系统业务逻辑的核心，包含 workflow 执行聚合、 workflow 图结构、知识文档 与用户等核心领域对象，并定义了向量检索、文件存储、文本切分、邮件服务等端口接口。领域层保持纯净，不依赖任何框架或外部库。

基础设施层：实现领域层定义的端口接口，包括节点执行策略、关系型数据库仓储适配、分布式缓存客户端、向量数据库适配、对象存储适配、邮件发送实现以及大模型框架适配。

前端基于 React 与 TypeScript 实现，包含 Agent 列表与画布编排页、对话执行页、人工审批页、知识库页与登录注册页等功能页面，并通过实时推送通道接收 workflow 执行事件。

系统功能设计

根据需求分析，系统围绕五个核心模块展开设计，功能结构如图 3.2 所示。



系统功能结构图

用户认证与系统安全支撑模块

该模块为整个平台提供统一的身份识别、访问控制与安全约束能力。系统支持邮箱注册、登录、找回密码与访问令牌认证，其中验证码发送、密码强度校验、密码强哈希存储与登录失败限制 均在认证领域服务中实现。认证成功后，接口层在请求上下文中解析用户身份，业务模块 据此执行 Agent 所有权校验、知识库访问控制、审批操作记录等逻辑。

在数据流层面，用户注册或登录成功后系统签发访问令牌，前端在后续请求中统一携带；接口层解析令牌并写入当前用户上下文，应用服务据此进行资源所有权校验：例如 Agent 更新需校验是否属于当前用户、知识文档上传需校验数据集归属、人工审批记录需保存当前审批人标识。因此，用户认证与系统安全并非孤立模块，而是贯穿其他四个业务模块的基础支撑层。

Agent 管理与可视化编排模块

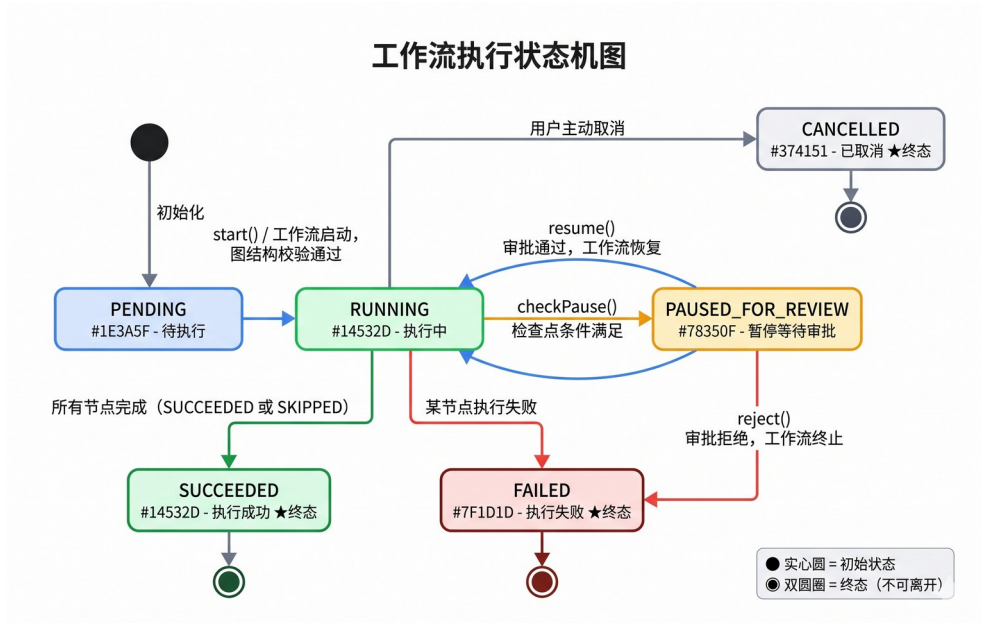
该模块负责 Agent 的全生命周期管理与可视化流程定义。后端的 Agent 应用服务 实现创建、编辑、发布、版本查询与回滚等能力，每个 Agent 对应一份以图结构形式存储的工作流定义；前端通过 ReactFlow 画布提供拖拽式编排界面，支持节点 创建、删除、参数配置与边连接，并将画布状态统一序列化为图结构数据。在发布流程中，系统会将当前草稿快照写入版本表并更新 Agent 的已发布版本号，保证开发中的草稿与线上执行版本相互隔离。

在数据流层面，用户在前端画布完成 Agent 编排后，节点与边被序列化为图结构数据；保存草稿时由后端校验图结构合法性，发布时生成版本快照并固化为可执行版本，运行时调度器再依据 Agent 标识与版本号读取对应的图定义作为工作流执行的输入，形成“画布编排 → 草稿保存 → 版本发布 → 运行时执行”的完整链路。

工作流调度执行模块

工作流执行模块以调度服务与工作流执行聚合的协作为核心。系统在执行开始时读取 Agent 的图定义，将其反序列化为内存中的工作流图结构，随后初始化执行上下文、节点 状态与就绪队列。执行引擎基于图结构的拓扑顺序与事件驱动调度，支持无依赖节点的 并行执行、条件分支、路径剪枝以及执行状态的整体推进。整个执行过程通过实时推送通道 持续向前端发送节点启动、完成、错误以及暂停恢复等事件。

执行状态机如图 3.3 所示。



工作流执行状态机

执行状态间的转换关系与触发条件如表 3.1 所示。

执行状态转换说明

源状态	目标状态	触发条件
PENDING	RUNNING	工作流启动成功，图结构初始化完成
RUNNING	SUCCEEDED	所有节点均完成（执行成功或被跳过）
RUNNING	FAILED	某节点执行失败
RUNNING	CANCELLED	用户主动取消
RUNNING	PAUSED_FOR_REVIEW	检查点触发，工作流进入人工审核状态
PAUSED_FOR_REVIEW	RUNNING	审批通过，工作流恢复执行
PAUSED_FOR_REVIEW	FAILED	审批拒绝，工作流终止

人工审核与恢复机制模块

该模块在工作流执行链路中植入“暂停 — 等待 — 恢复 / 终止”的控制能力。节点可配置人工检查点的开关与触发阶段，支持在节点执行前或执行后两种时机暂停工作流。设计上，系统采用分布式缓存维护待审批队列，并为每个执行实例配置分布式锁；审批人通过审批列表接口查看待处理任务，进入详情后可查看当前节点的上下文、修改中间结果并提交通过或拒绝操作。恢复或拒绝请求均以执行版本号作为并发控制条件，从而避免多人并发审批时出现状态覆盖问题。

知识库与检索增强模块

该模块为工作流中的知识节点与大模型节点提供外部知识支撑。从设计上看，知识链路由数据集管理、文档管理、异步处理与语义检索四部分组成。用户上传文档后，系统先将原始文件存入对象存储，再由异步处理器完成文档解析、文本分块与向量化入库；向量数据与原始文档元信息通过文档标识、数据集标识等关联字段建立联系，便于后续的删除、筛选与语义搜索。

在数据流层面，文档从上传到可检索的链路为：接口层校验文件 → 应用层创建文档记录并触发异步处理 → 基础设施层完成解析、分块、嵌入向量并写入向量数据库；运行时检索链路为：工作流知识节点拼接查询 → 嵌入模型生成查询向量 → 向量数据库语义检索 → 召回片段注入大模型上下文。这一双向链路保证了知识接入与运行时检索形成完整的闭环。

技术选型

为支撑上述系统结构与功能模块的实现，本系统在后端运行环境、AI 应用框架、数据存储、知识检索与前端等关键维度上进行了技术选型，主要选型与依据如表 3.2 所示。

系统关键技术选型

类别	选型	选型依据
后端运行环境	Java 21 + Spring Boot 3.x	虚拟线程提升并发能力，企业级生态成熟
AI 应用框架	Spring AI	统一的大模型调用接口，与 Spring 生态原生集成
数据存储	MySQL + Redis	业务持久化与分布式缓存、限流、并发控制
知识检索	Milvus + MinIO	私有化部署，支持语义

类别	选型	选型依据
前端	React + TypeScript + ReactFlow	检索与原始文件存储 类型安全、组件化开发、成熟的可视化画布

智能体编排系统详细设计与实现

本章在概要设计的基础上，对系统核心功能模块的详细实现进行说明。参照“实现过程—关键代码—界面展示”的组织方式，以下各节围绕用户认证、Agent 管理、 workflow 调度、人工检查点、知识库检索增强以及开发难点展开。每个功能模块先说明其在业务流程中的职责和前后端交互过程，再选取关键服务或领域对象中的核心代码片段进行说明，最后结合页面截图描述用户操作路径、状态变化和界面反馈，从而体现系统功能从后端逻辑到前端落地的完整闭环。

用户认证与系统安全支撑实现

注册、登录与密码找回流程

用户认证模块是系统对外开放访问的基础支撑，主要围绕邮箱验证码、注册、登录与找回密码形成闭环。该模块前端对应登录注册页面，后端由认证接口、用户应用服务、用户领域服务、验证码仓储和邮件服务共同完成。实现过程中，前端负责表单输入、格式校验、按钮状态控制和错误提示；后端负责验证码生成、密码强度校验、用户唯一性校验、密码哈希存储和登录失败限流。系统在验证码发送前校验邮箱格式和发送频率，验证码写入缓存后通过邮件服务发送给用户。

注册时，用户先输入邮箱并获取验证码，再填写用户名、密码和确认密码。后端依次校验验证码、密码强度以及邮箱和用户名唯一性，校验通过后使用 BCrypt 保存密码哈希。登录时，系统根据邮箱查询用户并校验密码，失败时记录限流信息，成功后清理失败计数并交由接口层签发令牌。找回密码流程复用验证码校验机制，在新密码符合强度要求后完成密码重置。该实现将“验证码有效性、密码安全性、登录限流和令牌签发”拆分到不同职责对象中，使认证流程既能支撑页面交互，也便于后续扩展短信验证码或第三方登录。

邮箱验证码发送、注册与登录的关键逻辑如下。

```
public void sendVerificationCode(String emailStr) {
    Email email = Email.of(emailStr);
    validateEmailSendRateLimit(email);
    String code = generateSecureCode();
    verificationCodeRepository.save(email.getValue(), code, 5);
}
```

```

        emailService.sendVerificationCode(email, code);
    }

    public User register(String emailStr, String code, String password, String
username) {
        Email email = Email.of(emailStr);
        validateVerificationCode(email, code);
        validatePasswordStrength(password);
        User user = User.create(email, Username.of(username),
BCryptUtil.encode(password));
        userRepository.save(user);
        verificationCodeRepository.delete(email.getValue());
        return user;
    }

    public User login(String emailStr, String password, String ip) {
        checkLoginRateLimit(emailStr, ip);
        User user = userRepository.findByEmail(Email.of(emailStr)).orElseThrow();
        if (!BCryptUtil.matches(password, user.getPasswordHash())) {
            recordLoginFailure(emailStr, ip);
            throw new IllegalArgumentException("密码错误");
        }
        clearLoginFailures(emailStr, ip);
        return user;
    }
}

```

异步邮件发送实现如下。

```

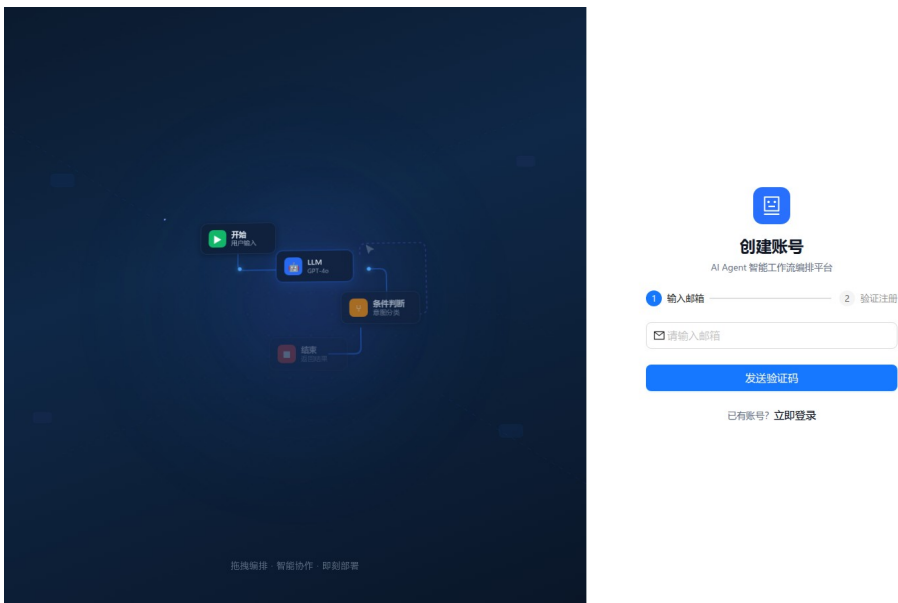
@Override
public void sendVerificationCode(Email to, String code) {
    try {
        doSendAsync(to, code);
    } catch (Exception e) {
        throw new RuntimeException("发送验证码失败", e);
    }
}

@Async
public void doSendAsync(Email to, String code) {
    doSend(to, code);
}

```

界面展示方面，用户进入系统后首先访问认证页面。页面提供登录、注册和找回密码三个入口，同一页面通过标签或表单切换承载不同认证操作，避免用户在多个页面之间反复跳转。注册与找回密码均需要先填写邮箱并点击“发送验证码”按钮，前端在验证码发送成功后展示倒计时，倒计时结束前禁止重复发送，同时对邮箱格式、密码长度和确认密码一致性进行表单校验。

当用户提交登录表单后，前端将邮箱和密码发送至认证接口，并在请求期间将登录按钮置为加载状态。若认证成功，系统返回访问令牌和用户基本信息，前端保存令牌后跳转到平台首页；若认证失败，页面会在表单下方给出错误提示，用户可重新输入或进入找回密码流程。对于验证码错误、密码强度不足、邮箱已注册等场景，页面均以明确文字提示用户修正输入。该模块保证了系统中的 Agent 管理、工作流执行、知识库管理等功能均运行在已认证用户上下文中。用户认证界面如图 4.1 所示。



用户认证界面

Agent 管理与可视化编排实现

Agent 创建与版本管理

Agent 管理模块承担工作流定义的生命周期管理职责，包括创建、修改、发布、回滚与版本查询。该模块的实现目标不是单纯保存一个 Agent 名称，而是将前端可视化画布中的节点、边、节点参数和人工审核配置统一组织为可执行的图定义。用户创建 Agent 时，系统根据默认模板生成初始 graphJson；用户在 React Flow 画布中完成节点拖拽、参数配置和连线后，前端将新的图结构提交到后端保存为草稿。

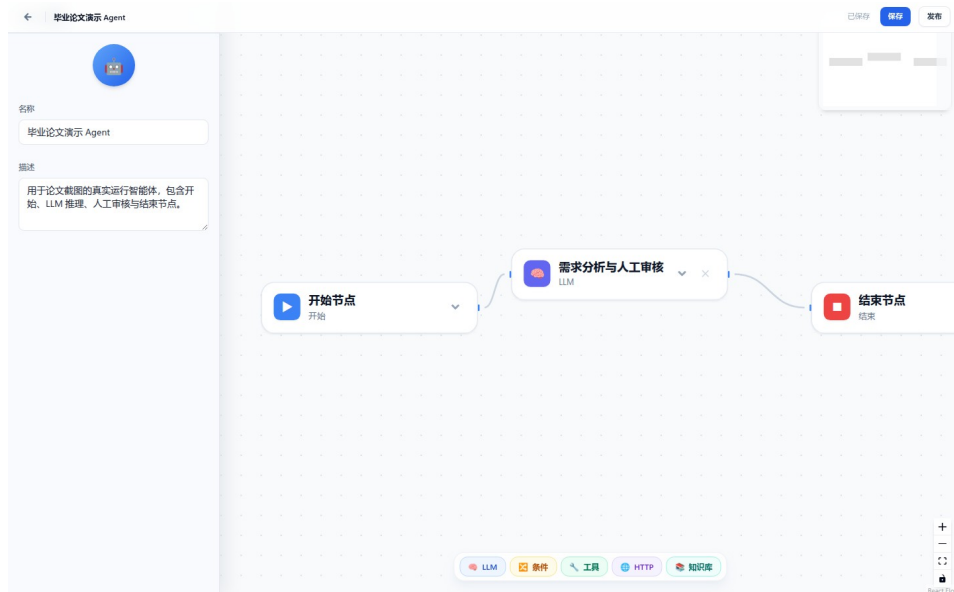
实现过程中，草稿保存只更新 Agent 当前编辑态配置，不影响已经发布的可执行版本；发布操作会结合图校验器检查节点连通性、环路和必要参数，然后生成 AgentVersion 快照并更新当前已发布版本指针。若后续需要恢复历史配置，系统可根据版本号读取快照并回滚当前 Agent。该机制形成了"草稿编辑 → 发布快照 → 历史回滚"的版本管理链路，保证运行时图定义具备可追溯性，也避免用户正在编辑的草稿影响线上执行。

Agent 发布版本快照时，系统基于当前草稿构造版本快照并更新已发布指针，其核心逻辑如下。

```
@Transactional(rollbackFor = Exception.class)
public void publishAgent(PublishAgentCmd cmd) {
    Agent agent = agentRepository.findById(cmd.getId()).orElseThrow();
    Integer nextVersion =
agentRepository.findMaxVersion(agent.getId()).orElse(0) + 1;
    AgentVersion version = agent.publish(graphValidator, nextVersion);
    agentRepository.saveVersion(version);
    agent.setPublishedVersionId(version.getVersion().longValue());
    agentRepository.save(agent);
}
```

界面展示方面，Agent 编排页面面向应用构建者，是系统中最核心的配置界面。页面通常由三部分组成：左侧为节点组件区，展示 LLM、条件判断、知识检索、工具调用、人工检查点等可拖拽节点；中间为 React Flow 画布，用于放置节点、连接边并展示流程结构；右侧为属性配置面板，用于编辑当前选中节点的名称、输入输出字段、模型参数和人工审核配置。

用户创建 Agent 后进入该页面，先填写 Agent 名称、描述等基础信息，再从左侧拖入节点并在画布中建立连接关系。当用户选中某个节点时，右侧属性面板会根据节点类型动态展示配置项，例如 LLM 节点展示模型、提示词和输出字段，条件节点展示判断表达式和分支映射，知识节点展示数据集和召回数量。当用户点击“保存草稿”时，前端将画布中的节点、边和节点参数序列化为 graphJson 提交给后端保存；当用户点击“发布版本”时，系统会先执行图结构校验，校验通过后生成可执行版本。如果存在必填参数缺失、连线不完整或流程图存在环路，页面会以错误提示的方式引导用户继续修改，并保持当前画布内容不丢失。Agent 可视化编排界面如图 4.2 所示。



Agent 可视化编排界面

工作流图定义与运行时衔接

Agent 可视化编排的最终结果并不是静态页面数据，而是运行时调度的直接输入。当前端保存画布后，节点、边、节点参数等结构统一写入 graphJson 字段。该字段既作为编辑态草稿保存，也可在发布后固化为版本快照。

工作流执行启动时，调度器首先根据 agentId 查询 Agent 实体。若用户显式指定了 versionId，则优先读取对应 AgentVersion 的图定义；否则优先加载已发布版本，若不存在再回退到当前草稿。随后，调度器通过图工厂将 graphJson 反序列化为领域层的 WorkflowGraph，供后续 Execution 聚合根执行调度。

这一机制实现了可视化编排与运行时执行的直接衔接，使前端画布不只是配置界面，而是工作流引擎的图定义来源。由于执行时优先读取已发布版本，用户在画布中继续修改草稿不会影响已经启动的执行任务，从而保证配置迭代和流程运行之间的隔离。

调度器读取版本图定义并反序列化为工作流图的关键过程如下。

```
public void startExecution(String executionId, Long agentId, Long userId,
    String conversationId, Integer versionId,
    Map<String, Object> inputs, ExecutionMode mode) {
    Agent agent = agentRepository.findById(agentId).orElseThrow();

    String graphJson;
    if (versionId != null) {
        graphJson = agentRepository.findVersion(agentId, versionId)
            .orElseThrow().getGraphJson();
    } else if (agent.getPublishedVersionId() != null) {
```

```

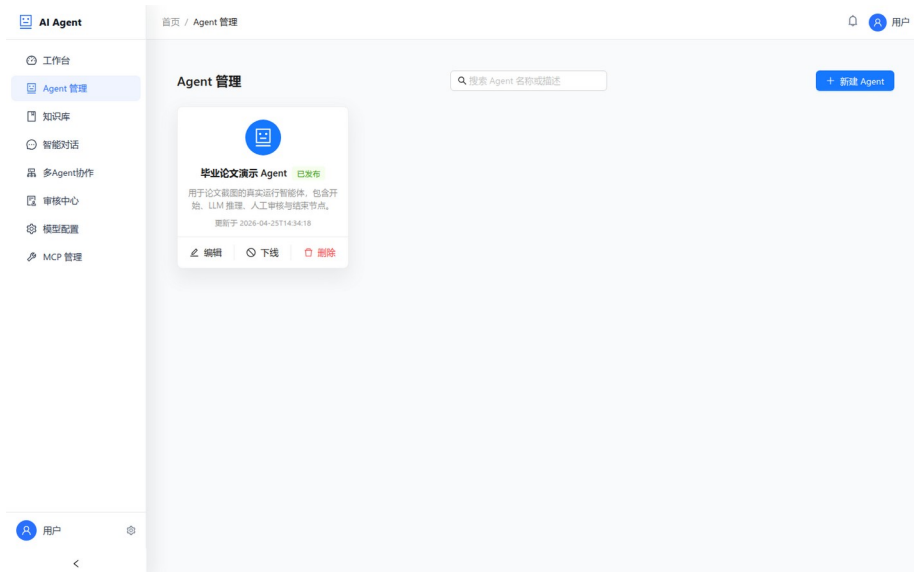
        graphJson = agentRepository.findVersion(agentId,
            agent.getPublishedVersionId().intValue())
            .orElseThrow().getGraphJson();
    } else {
        graphJson = agent.getGraphJson();
    }

    WorkflowGraph graph = workflowGraphFactory.fromJson(graphJson);
    Execution execution = Execution.create(executionId, agentId, userId,
        conversationId, graph, mode);
    startExecution(execution, inputs, mode);
}

```

Agent 列表页用于集中展示应用构建者已经创建的智能体应用。页面以表格形式展示 Agent 名称、描述、创建时间、更新时间、当前状态和已发布版本等信息，每一行提供编辑、运行、发布、版本历史和删除等操作入口。用户点击“编辑”后进入可视化画布继续修改草稿，点击“运行”则使用当前已发布版本启动执行。

版本管理界面用于解决 workflow 配置频繁迭代带来的追溯问题。用户点击“版本历史”后，可以查看每次发布生成的版本号、发布时间和发布说明；当某个新版本配置错误时，用户可选择历史版本并点击“回滚”，前端提交回滚请求后刷新列表中的当前发布版本指针。通过该界面，系统将 Agent 的创建、发布、运行和回滚操作组织为完整的管理闭环。当回滚成功后，前端会刷新当前发布版本号和更新时间；当删除或发布失败时，页面保留原列表状态并展示失败原因，避免误导用户认为操作已经生效。Agent 列表与版本管理界面如图 4.3 所示。



Agent 列表与版本管理界面

workflow 调度执行实现

workflow 执行启动模块

workflow 执行的启动入口位于应用层调度服务中。调度器根据 Agent 编号与可选的版本号读取对应图定义，再通过图工厂将 JSON 配置转换为领域层 workflow 图对象。

实现过程中，用户在前端点击“开始执行”后，接口层创建执行编号并调用调度服务。执行启动前，系统会向 `ExecutionContext` 注入会话历史和向量检索结果，随后调用 `Execution.start(inputs)` 完成 DAG 校验、节点状态初始化和首批就绪节点计算。初始执行状态持久化后，调度器异步提交首批节点执行任务， workflow 由此进入运行阶段。

workflow 执行启动的核心逻辑如下。

```
private void startExecution(Execution execution,
    Map<String, Object> inputs, ExecutionMode mode) {

    // 上下文增强：加载会话历史和检索结果
    hydrateMemory(execution, inputs);

    // 启动执行聚合根，获取首批就绪节点
    List<Node> readyNodes = execution.start(inputs);

    // 持久化初始状态并调度首批节点
    executionRepository.save(execution);
    scheduleNodes(execution.getExecutionId(), readyNodes, null);
}
```

界面展示方面，业务使用者在前端选择已发布的 Agent 后，可以填写运行输入并点击“开始执行”按钮。前端提交启动请求后，会根据返回的执行编号建立 SSE 连接，持续接收节点开始、节点完成、节点失败、 workflow 暂停和 workflow 结束等事件。执行页面以画布形式复用 Agent 图结构，使用户能够直观看到任务在各节点之间的推进过程。

界面中不同颜色表示不同执行状态：蓝色表示节点正在运行，绿色表示节点执行成功，红色表示节点执行失败，灰色表示条件分支中未被选中的路径被跳过。当用户点击某个节点时，右侧详情区域会展示该节点的输入、输出、耗时和错误信息；下方日志区域按时间顺序追加执行事件，便于用户在执行失败时定位具体节点和原因。执行完成后，页面展示最终输出和总耗时；若执行进入人工审核等待状态，页面会在画布和日志中同时提示当前暂停节点，引导用户进入审批页面处理。 workflow 执行界面如图 4.4 所示。



workflow 执行界面

DAG 调度核心模块

Execution 是 workflow 领域的聚合根，负责管理节点状态、执行上下文和整体状态流转。该模块的实现重点在于把“节点完成事件”转化为“后续可执行节点集合”，并在同一位置处理条件分支、失败传播、人工暂停和整体完成等状态变化。每个节点完成后，调度器调用 `advance(nodeId, result)` 写入节点状态与输出，并根据结果判断是否需要条件分支剪枝、人工检查点暂停或继续推进后继节点。

在计算后继节点时，`getReadyNodes()` 会结合有效入度判断可运行节点；已完成、已跳过或已失败的节点不再阻塞其后继节点，因此系统可以自动调度所有依赖已满足的节点并行执行。

`Execution.advance()` 的完整逻辑如下。

```
public List<Node> advance(String nodeId, NodeExecutionResult result) {
    nodeStatuses.put(nodeId, result.getStatus());
    if (result.getOutputs() != null) {
        context.setNodeOutput(nodeId, result.getOutputs());
    }
    if (result.isRouting()) {
        pruneUnselectedBranches(nodeId, result.getSelectedBranchId());
    }
    if (result.isPaused()) {
        this.status = ExecutionStatus.PAUSED_FOR_REVIEW;
        this.pausedNodeId = nodeId;
        this.pausedPhase = result.getTriggerPhase();
        this.version++;
        return Collections.emptyList();
    }
    this.version++;
    if (isCompleted()) {
        this.status = ExecutionStatus.SUCCEEDED;
    }
}
```

```

        return Collections.emptyList();
    }
    if (hasFailed()) {
        this.status = ExecutionStatus.FAILED;
        return Collections.emptyList();
    }

    return getReadyNodes();
}

```

条件分支剪枝的递归逻辑如下。

```

private void skipNodeRecursively(String nodeId) {
    if (nodeStatuses.get(nodeId) != ExecutionStatus.PENDING) { return; }
    nodeStatuses.put(nodeId, ExecutionStatus.SKIPPED);

    Set<Node> successors = graph.getSuccessors(nodeId);
    for (Node successor : successors) {
        Set<Node> predecessors =
graph.getPredecessors(successor.getNodeId());
        if (predecessors.size() <= 1) {
            skipNodeRecursively(successor.getNodeId());
        } else {
            boolean allSkipped = predecessors.stream().allMatch(
                pred -> nodeStatuses.get(pred.getNodeId())
                    == ExecutionStatus.SKIPPED);
            if (allSkipped) {
                skipNodeRecursively(successor.getNodeId());
            }
        }
    }
}
}

```

界面展示方面， workflow 执行过程中，前端会将每个节点的运行日志集中展示在日志面板中。

日志内容按照时间顺序排列，包含节点名称、节点类型、开始时间、结束时间、输入参数、输出结果以及错误信息等字段。对于 LLM 节点，页面还会展示模型调用使用的关键提示词片段和生成结果；对于条件节点，页面会展示分支判断结果和被跳过的后继路径。

用户在画布中点击任意节点后，日志面板会自动筛选该节点相关记录，便于从整体执行视图切换到单个节点的细节视图。当 workflow 执行失败时，用户可根据红色失败节点和日志中的异常信息定位问题，从而判断是参数配置错误、模型调用异常还是知识检索结果为空。这种“画布状态 + 日志明细 + 节点详情”的组合展示方式，使 workflow 执行过程不再是黑盒运行，而是可以被业务使用者和应用构建者共同观察和复盘。

节点执行器策略模式

系统采用策略模式解耦各节点类型的执行逻辑，定义统一的节点执行器接口，每种节点类型（LLM、CONDITION、KNOWLEDGE、TOOL、START、END）各自实现一个策略类，并注册到执行器工厂中统一管理。

实现过程中，调度器在收到待执行节点后，从工厂中根据节点类型获取对应策略，调用其异步执行接口完成节点逻辑。新增节点类型只需实现接口并注册，无需修改调度核心代码，满足开闭原则。

```
public interface NodeExecutorStrategy {
    CompletableFuture<NodeExecutionResult> executeAsync(
        Node node,
        Map<String, Object> resolvedInputs,
        StreamPublisher streamPublisher);

    NodeType getSupportedType();
}
```

人工检查点与 workflow 恢复机制

人工检查点触发模块

人工检查点（Human Checkpoint）用于在高风险节点执行前后引入人工审批。该模块是本文系统区别于普通自动化工作流的重要实现点，其目标是在自动执行链路中增加可控闸门，使模型输出或高风险操作能够在人工确认后再进入下一步。调度器在节点执行前后调用 checkPause()，根据节点配置、触发阶段和已审核标记判断是否需要暂停。

实现过程中，当检查点条件满足时，系统在分布式锁保护下将执行状态更新为 PAUSED_FOR_REVIEW，记录暂停节点和触发阶段，并将执行编号写入 Redis 待审批队列。随后系统通过 SSE 推送 workflow_paused 事件，前端据此展示审批弹窗。

系统支持两种触发阶段，其语义差异如表 4.1 所示。

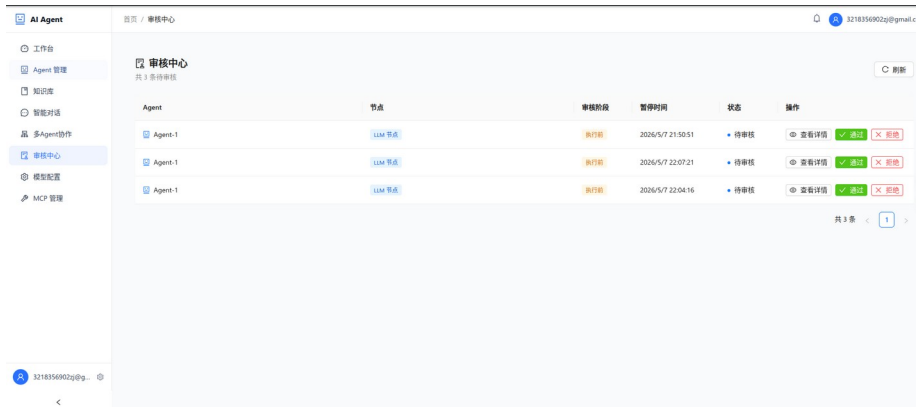
人工检查点两种触发阶段对比

维度	BEFORE_EXECUTION	AFTER_EXECUTION
触发时机	节点执行前	节点执行后
节点是否已执行	否	是
审批人可修改内容	节点输入（ <code>edits</code> → <code>SharedState</code> ）	节点输出（覆盖 <code>nodeOutput</code> ）
审批通过后节点行为	重新执行节点	跳过执行，直接推进后

维度	BEFORE_EXECUTION	AFTER_EXECUTION
		续节点
典型场景	高风险操作预审（发送邮件前）	AI 输出内容人工校正
<pre> private boolean checkPause(String executionId, Node node, TriggerPhase phase, StreamPublisher publisher, Map<String, Object> outputs) { HumanReviewConfig config = node.getHumanReviewConfig(); if (config == null !config.isEnabled()) { return false; } if (config.getTriggerPhase() != phase) { return false; } Execution execution = executionRepository.findById(executionId).orElseThrow(); if (execution.isNodeReviewed(node.getNodeId())) { return false; } RLock lock = redisService.getLock("lock:exec:" + executionId); try { lock.lock(30, TimeUnit.SECONDS); execution.advance(node.getNodeId(), NodeExecutionResult.paused(phase, outputs)); executionRepository.update(execution); humanReviewQueuePort.addToPendingQueue(executionId); publisher.publishEvent("workflow_paused", Map.of(...)); } finally { if (lock.isHeldByCurrentThread()) { lock.unlock(); } } return true; } </pre>		

界面展示方面，当 workflow 执行到配置了人工检查点的节点时，前端通过 SSE 接收 workflow_paused 事件，弹出审批面板。界面展示节点的输入（BEFORE 阶段）或输出（AFTER 阶段）内容，审批人可视情况对字段进行修改，然后点击“通过”或“拒绝”。审批面板上方展示当前执行编号、暂停节点、触发阶段和提交时间，中间区域展示待审核内容，底部提供审批意见输入框和操作按钮。

如果审批人选择“通过”，前端会将审批意见和可选修改内容一并提交给后端；如果选择“拒绝”，页面要求填写拒绝原因，并在提交后提示该 workflow 将被终止。在审批请求提交期间，按钮会进入加载状态，避免用户重复点击；若后端返回版本冲突或该任务已被其他审批人处理，页面会提示用户刷新审批状态。通过该界面，审批人可以在不理解底层 DAG 调度细节的情况下完成“查看上下文—修正中间结果—通过或拒绝”的业务操作。人工检查点审批界面如图 4.5 所示。



人工检查点审批界面

workflow恢复模块

审批通过后，前端调用 `POST /api/workflow/reviews/resume` 接口并携带执行编号、暂停节点、乐观锁版本号以及可选编辑内容。服务端在分布式锁内校验暂停节点和版本号，防止重复审批或并发恢复导致状态不一致。

实现过程中，系统会根据暂停阶段合并审批人修改的数据，写入审核记录，调用 `execution.resume()` 将 workflow 恢复为运行态，并从 Redis 待审批队列移除该执行。若暂停发生在节点执行后，调度器直接推进后继节点；若暂停发生在节点执行前，则重新调度当前节点执行。

```
public void resumeExecution(String executionId, String nodeId, Integer expectedVersion,
    Map<String, Object> edits, Long reviewerId, String comment) {
    RLock lock = redisService.getLock("lock:exec:" + executionId);
    try {
        lock.lock(30, TimeUnit.SECONDS);
        Execution execution =
            executionRepository.findById(executionId).orElseThrow();
        if (!pausedNode(execution, nodeId)) { return; }
        if (expectedVersion != null && !
            expectedVersion.equals(execution.getVersion())) {
            throw new OptimisticLockingFailureException("版本冲突");
        }
        TriggerPhase phase = defaultPhase(execution.getPausedPhase());
        mergeReviewEdits(execution, nodeId, edits, phase);
        humanReviewRepository.save(buildApproveRecord(executionId, nodeId,
            reviewerId, phase, comment));

        List<Node> readyNodes = execution.resume(nodeId, edits);
        executionRepository.update(execution);
        humanReviewQueuePort.removeFromPendingQueue(executionId);
        publishAndSchedule(executionId, nodeId, readyNodes, phase);
    }
```

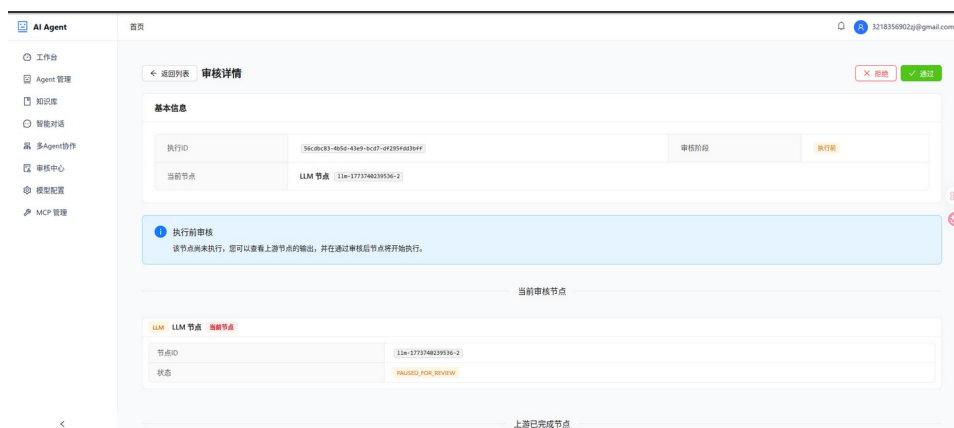
```

    } finally {
        if (lock.isHeldByCurrentThread()) { lock.unlock(); }
    }
}

```

界面展示方面，在审批人提交通过或拒绝操作之前，审批详情页会展示当前待审核执行的基本信息，包括执行编号、当前节点、审核阶段和节点状态等内容。当人工检查点配置在节点执行前时，页面会提示该节点尚未执行，审批人可以查看上游节点输出和当前节点输入，确认是否允许后续节点继续执行。

该界面右上角提供“通过”和“拒绝”按钮，用于提交人工审核决策。如果审批人点击“通过”，系统才会进入恢复流程，并根据暂停阶段重新调度当前节点或推进后继节点；如果点击“拒绝”，前端会接收终止事件，将 workflow 状态标记为失败或已拒绝。因此，该截图反映的是 workflow 处于 PAUSED_FOR_REVIEW 状态时的准备审核界面，而不是审批通过后的恢复执行结果。人工检查点准备审核界面如图 4.6 所示。



人工检查点准备审核界面

知识库与检索增强实现

知识数据集与文档接入流程

知识库模块用于为 workflow 中的知识节点和 LLM 节点提供外部知识支撑。该模块的实现目标是让应用构建者能够把产品文档、业务制度、操作手册等非结构化资料接入系统，并在 workflow 运行时通过语义检索为模型生成提供上下文。接口层通过 KnowledgeController 提供数据集创建、文档上传、查询、删除和检索接口；应用层由 KnowledgeApplicationService 串联文件校验、对象存储、文档记录入库和异步处理。

实现过程中，用户上传文档时，系统先完成数据集归属校验和文件合法性校验，再将原始文件保存到 MinIO，同时创建状态为 PENDING 的文档记录。随后应用层触发异步文档处理任务，使上传接口可以快速返回，避免解析和向量化等耗时操作阻塞前端请求。

文档上传和异步处理入口的关键逻辑如下。

```
@Transactional
public KnowledgeDocument uploadDocument(Long userId, String datasetId,
    MultipartFile file, ChunkingConfig chunkingConfig) {
    KnowledgeDataset dataset =
datasetRepository.findById(datasetId).orElseThrow();
    validateOwnership(userId, dataset.getCreatedBy());

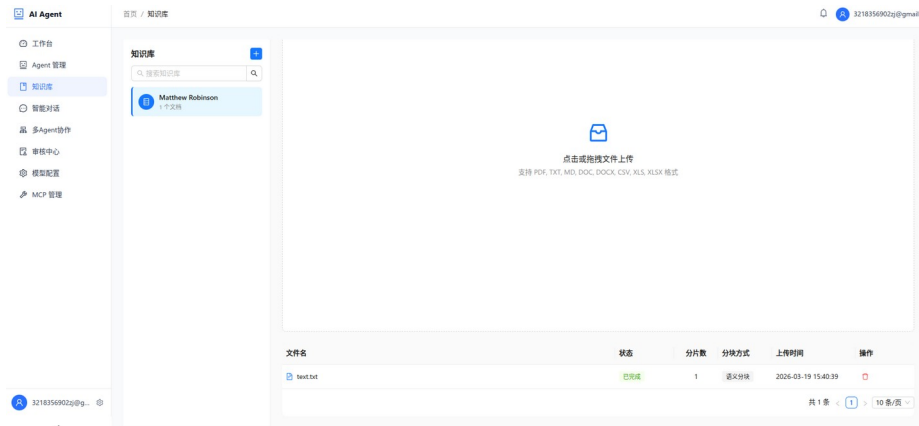
    fileValidator.validate(file);
    String fileUrl = fileStorageService.upload(bucketName, objectName,
        file.getInputStream(), file.getContentType());

    KnowledgeDocument document = KnowledgeDocument.pending(
        datasetId, file.getOriginalFilename(), fileUrl, chunkingConfig);
    documentRepository.save(document);

    dataset.addDocument();
    datasetRepository.save(dataset);
    asyncDocumentProcessor.processDocumentAsync(document);
    return document;
}
```

界面展示方面，知识库页面面向应用构建者，主要用于维护 Agent 执行所需的外部知识。页面左侧展示数据集列表，用户可以创建新的数据集，也可以切换不同数据集查看其文档内容；右侧展示当前数据集下的文档列表，包括文件名、文件类型、上传时间、处理状态和操作按钮。

用户点击“上传文档”后，前端弹出文件选择框，并在提交前校验文件类型和大小。上传成功后，文档会先以“待处理”状态显示在列表中；异步任务开始后状态变为“处理中”，处理完成后变为“已完成”，处理失败时显示失败原因。如果用户在处理完成前进入 Agent 编排页面引用该数据集，系统仍会展示数据集名称，但检索测试结果可能为空；因此页面通过状态列明确提示文档是否已经可检索。通过这种状态反馈，用户可以明确区分“文件已上传”和“知识已可检索”两个阶段，避免在文档尚未向量化完成时误认为知识库已经生效。知识库管理界面如图 4.7 所示。



知识库管理界面

异步分块、向量化与检索增强

异步文档处理器承担知识接入链路中的核心计算任务，整体流程为 "对象存储下载 → 文档解析 → 文本分块 → 批量向量化存储 → 状态回写"。处理器先将文档状态更新为 PROCESSING，再通过 DocumentReaderPort 解析 PDF、Word、Markdown 等文件，并按照 ChunkingConfig 调用 TextSplitterPort 完成文本分块。

实现过程中，分块完成后，系统为每个文本块构造向量文档和元数据，按批次写入 Milvus。运行时，工作流或检索接口可根据用户问题进行语义召回，并将召回结果作为知识上下文注入 LLM 节点输入，实现检索增强生成。

异步文档处理与批量向量化的关键实现如下。

@Async

```
public void processDocumentAsync(KnowledgeDocument document) {
    document.startProcessing();
    documentRepository.save(document);

    InputStream fileStream = fileStorageService.download(bucketName,
        extractObjectName(document.getFileUrl()));
    List<String> parsedTexts = documentReaderPort.readDocument(
        fileStream, document.getFilename());

    ChunkingConfig config = document.getChunkingConfig().normalized();
    List<String> chunks = textSplitterPort.split(parsedTexts, config);

    List<Document> batch = buildVectorDocuments(document, chunks);
    vectorStore.addDocuments(batch);

    document.markCompleted();
}
```

```
        documentRepository.save(document);
    }
}
```

基于向量数据库的检索增强接口如下。

```
public List<String> searchKnowledge(String query, Long agentId, int topK) {
    SearchRequest request = SearchRequest.query(query)
        .withTopK(topK)
        .withFilterExpression("agent_id == " + agentId);
    return springAiVectorStore.similaritySearch(request).stream()
        .map(Document::getText)
        .toList();
}
```

界面展示方面，在文档完成分块和向量化后，用户可以通过知识库页面的检索入口输入测试问题，前端将问题、数据集编号和召回数量提交给后端检索接口。检索结果以列表形式展示命中的文本片段、所属文档和相似度信息，便于用户判断知识库内容是否能够支撑后续 workflow 执行。

当 workflow 中的知识节点或 LLM 节点引用该数据集时，系统会将检索到的文本片段注入节点上下文中。因此，前端检索测试不仅用于展示数据集命中文档摘要，也用于帮助应用构建者在发布 Agent 前验证知识增强效果，避免因文档内容缺失、分块过粗或召回数量设置不合理而影响模型回答质量。

系统开发难点与解决方案

在系统实现过程中，workflow 调度、并发控制、实时推流等模块均遇到了若干工程难点。本节梳理五个典型难点，阐述问题根因与对应解决方案，以体现系统在关键路径上的设计深度。

DAG 环检测与死锁防护

workflow 引擎以 DAG 作为调度基础，如果用户在前端画布中无意创建了回路（如节点 $A \rightarrow B \rightarrow C \rightarrow A$ ），调度器在执行 `Execution.start()` 时就会陷入无限等待——所有节点相互依赖，入度永远无法降为 0，导致 workflow 永久挂起。

解决过程中，系统在 `start()` 调用时，在初始化节点状态之前优先执行 Kahn 拓扑排序：从入度为 0 的节点出发，逐步消除出边并将入度降为 0 的后继节点加入队列，直到所有节点处理完毕。若最终处理节点数小于图中总节点数，说明存在环路，立即抛出 `IllegalStateException`，并在接口层返回 HTTP 400。

```
public void validateAcyclic() {
    Map<String, Integer> inDegree = new HashMap<>();
    for (Node node : nodes.values()) {
        inDegree.put(node.getNodeId(), 0);
    }
}
```

```

    }
    for (Edge edge : edges) {
        inDegree.merge(edge.getTarget(), 1, Integer::sum);
    }

    Queue<String> queue = new LinkedList<>();
    inDegree.forEach((id, deg) -> { if (deg == 0) queue.offer(id); });

    int visited = 0;
    while (!queue.isEmpty()) {
        String cur = queue.poll();
        visited++;
        for (Node succ : getSuccessors(cur)) {
            int newDeg = inDegree.merge(succ.getNodeId(), -1, Integer::sum);
            if (newDeg == 0) queue.offer(succ.getNodeId());
        }
    }
    if (visited != nodes.size()) {
        throw new IllegalStateException(" workflow图存在环路, 无法执行");
    }
}

```

该检测在 `start()` 前执行，时间复杂度为 $O(V+E)$ ，对正常图无额外性能开销。

人工审批并发一致性保障

在多人协作场景下，同一暂停执行可能被多名审批人同时进入审批页面，若没有任何保护机制，则可能出现：（a）两次审批成功导致执行状态被转换两次；（b）先 approve 后 reject 导致结果不一致；（c）数据库更新覆盖彼此的版本号。

解决过程中，系统采用两层并发保护机制联合兜底：

第一层：Redis 分布式锁（Redisson RLock）。所有 `pause / resume / reject` 操作均在锁 `lock:exec:{executionId}` 保护下执行，超时设置为 30 秒，确保同一时刻只有一个线程进入临界区。

第二层：乐观锁（`version` 字段）。Execution 聚合根维护 `version` 自增字段，前端发起审批时携带当前 `expectedVersion`，服务端校验不一致时抛出乐观锁冲突异常，HTTP 层返回 409 Conflict，前端提示“操作已被他人完成，请刷新重试”。

该设计保证即使分布式锁短暂失效，乐观锁也能兜底阻断重复审批。在功能测试 TC-HR-06 中，通过模拟两路并发 approve 请求，验证后提交者必然收到 409。

BEFORE_EXECUTION 审批后防重复暂停

当人工检查点配置为 BEFORE_EXECUTION 时，审批通过后调度器会重新执行该节点（因为节点执行前被拦截，尚未运行）。如果不加额外控制，节点重新执行时 checkPause() 会再次判定满足条件，导致工作流陷入"无限暂停"的死循环。

解决过程中，系统在 Execution 聚合根中维护 reviewedNodes 集合，记录所有已完成审批的节点 ID。checkPause() 的首要判断为：

```
if (execution.isNodeReviewed(node.getNodeId())) {  
    return false; // 已审批过，直接跳过暂停  
}
```

resume() 时，系统将当前节点加入 reviewedNodes 并持久化，从而保证节点无论被重新执行多少次，都不会再触发同一审批检查点。

SSE 长连接稳定性与心跳保活

工作流执行时间可能较长（例如多节点串行、LLM 推理延迟），浏览器默认会在约 30-60 秒无数据传输后自动关闭 SSE 连接，导致前端画布停止更新，用户误认为系统卡死。此外，nginx 等反向代理也会因超时断开长连接。

解决过程中，系统实现了服务端主动心跳机制：在 SseEmitter 建立后，后台定时任务每 15 秒推送一次 ping 类型的 SSE 事件（携带当前服务端时间戳），使连接保持活跃状态直到执行完成或显式关闭。

同时，系统在 SseEmitter 的 onTimeout 和 onError 回调中完成资源清理（从 emitter 池移除、取消心跳任务），防止 emitter 泄漏积压。此方案在 SSE 推流延迟测试中验证了 100 次执行样本最大延迟 387ms，无连接中断记录。

文档向量化阻塞上传接口

知识库文档上传后需经历"PDF/Word 解析 → 文本分块 → 批量调用嵌入模型 → 写入 Milvus"的完整处理链路，对于一份 50 页的 PDF 文档，同步处理耗时可达数十秒，严重阻塞前端上传请求，造成超时或用户体验下降。

解决过程中，系统将文档处理链路设计为异步解耦模式：上传接口仅完成文件合法性校验、MinIO 存储和 PENDING 状态记录后立即返回，再通过 Spring @Async 触发异步处理器在独立线程中完成后续处理，并在处理完成后回写文档状态为 COMPLETED 或 FAILED。

前端通过轮询文档列表接口观察状态变化，实现非阻塞的文档接入体验。该设计使上传接口响应时间从同步场景的数十秒降至 500ms 以内（仅含文件校验和 MinIO 上传耗时），显著改善了大文档场景下的用户体验。

综合上述五类开发难点的应对方案，主要难点与解决方案汇总如表 4.2 所示。

主要开发难点与解决方案汇总

难点	根因	解决方案
DAG 环路死锁	回路导致入度永不为 0	Kahn 拓扑排序， $O(V+E)$ 复杂度
并发审批冲突	多人同时操作暂停执行	Redis 分布式锁 + 乐观锁双保险
BEFORE 审批重复暂停	resume 后节点重新执行	reviewedNodes 集合 标记已审节点
SSE 连接超时断开	浏览器/代理 idle 超时	服务端 15s 心跳 ping 事件
向量化阻塞上传	处理链路耗时长	@Async 异步解耦 + 状态回写

本章小结

本章按照“实现过程—关键代码—界面展示”的思路，对智能体编排系统的核心功能实现进行了说明。用户认证模块实现了邮箱验证码、注册登录、密码找回和令牌访问控制，为后续业务操作提供身份基础；Agent 管理模块通过草稿、发布版本和历史回滚机制，将 React Flow 画布配置转化为可追溯的可执行图定义；工作流调度模块基于 Execution 聚合根完成 DAG 节点推进、条件分支剪枝和节点执行器分发；人工检查点模块在自动执行链路中引入暂停、审批、修改、恢复和拒绝终止机制，使系统具备关键节点人工把关能力；知识库模块通过文档上传、异步分块、向量化入库和语义检索，为 LLM 节点提供外部知识增强。最后，本章还总结了 DAG 环检测、并发审批一致性、重复暂停防护、SSE 心跳保活和文档异步处理等开发难点及解决方案。通过上述实现，系统形成了从可视化配置、自动执行、人工审核到知识增强的核心功能闭环。

智能体编排系统测试与部署

测试环境配置

软硬件环境

测试所采用的软硬件环境配置如表 5.1 所示。

测试环境配置

项目	配置
操作系统	Ubuntu 22.04 LTS
JDK	OpenJDK 21（含虚拟线程支持）
Spring Boot	3.x
Spring AI	最新稳定版
MySQL	8.0.32（Docker 容器）
Redis	7.0（Docker 容器）
Milvus	2.3.x（Docker 容器）
MinIO	最新稳定版（Docker 容器）
浏览器	Chrome 123
前端 Node.js	18.x

测试工具

- 单元测试：JUnit 5 + Mockito（验证调度器、认证、知识处理等核心逻辑）
- 集成测试：Spring Boot Test（验证跨层调用和数据库读写）
- 接口测试：Postman / curl（验证 REST 接口及 SSE 推流）
- 性能测试：Apache JMeter 5.x（并发压测）
- 前端功能验证：浏览器页面交互 + DevTools Network 面板（验证画布、审批与 SSE 事件流）

单元测试

系统使用 JUnit 5 + Mockito 框架，针对调度器、Agent 生命周期、知识处理和用户认证等核心领域逻辑共编写 42 个单元测试用例，核心领域层代码行覆盖率达 78%，分支覆盖率达 71%。测试重点覆盖以下边界场景：

- DAG 环检测：含环的工作流图启动时应返回“流程图存在环路”错误，工作流被阻断
- 就绪节点计算：给定节点状态集合，调度器应正确识别可执行节点集合

- 条件分支剪枝：选定分支后未选路径上的所有后继节点应被标记为 SKIPPED
- 防重复暂停：节点完成审批后再次执行时应跳过暂停判断，正常完成
- 乐观锁校验：执行版本号不匹配时应返回 409 Conflict，阻断重复审批
- Agent 发布与回滚：版本快照生成、已发布版本指针更新与回滚逻辑正确
- 知识文档处理：文档状态在 PENDING → PROCESSING → COMPLETED / FAILED 之间正确流转
- 用户认证：验证码校验、密码强度校验、登录失败限制与密码哈希匹配逻辑正确

所有 42 个测试用例均已通过，无失败或跳过项。

系统功能测试

采用黑盒测试方法，围绕五个核心模块共设计 35 个功能测试用例，覆盖 Agent 管理、 workflow 执行、人工检查点、知识库与用户认证全部业务场景。为避免将全部测试用例逐条展开造成表格过长，本文按模块给出测试覆盖汇总，并选取能够代表关键业务链路和异常分支的用例进行列示。各模块测试覆盖情况如表 5.2 所示。

系统功能测试覆盖汇总

测试模块	用例数量	覆盖重点	结果
Agent 管理	5	创建、画布保存、版本发布、历史回滚、权限校验	全部通过
工作流执行	7	执行启动、SSE 推流、条件分支、并行节点、取消执行、日志查询	全部通过
人工检查点	9	BEFORE / AFTER 暂停、审批通过、结果修改、审批拒绝、并发冲突、历史查询	全部通过

测试模块	用例数量	覆盖重点	结果
知识库	6	数据集创建、文档上传、异步处理、解析失败、语义检索、知识节点调用	全部通过
用户认证	8	邮箱验证码、注册登录、密码错误、登录限流、找回密码、令牌拦截	全部通过
合计	35	覆盖系统五个核心模块及主要异常分支	全部通过

代表性功能测试用例如表 5.3 所示。该表重点列出与系统核心能力直接相关的主流程、人工审核分支和异常处理场景，其余同类用例按照相同方法完成验证。

代表性功能测试用例

编号	模块	测试场景	预期结果	结果
TC-AG-03	Agent 管理	发布 Agent 版本	生成版本快照，已发布版本指针更新	通过
TC-WF-01	workflow 执行	启动标准 workflow	SSE 推送连接事件与节点状态事件	通过
TC-WF-03	workflow 执行	条件分支路由	正确选中分支，未选中路径标记为 SKIPPED	通过
TC-HR-01	人工检查点	BEFORE_EXECUTION 暂停	推送 workflow_paused，并加入待审批队列	通过

编号	模块	测试场景	预期结果	结果
TC-HR-04	人工检查点	审批通过并修改输出	修改值写入上下文，后续节点使用新结果	通过
TC-HR-06	人工检查点	并发审批冲突	后提交请求被拒绝，返回 409 Conflict	通过
TC-KN-02	知识库	上传文档并异步处理	文档状态按 PENDING、PROCESSING、COMPLETED 流转	通过
TC-AU-08	用户认证	未携带令牌访问接口	请求被拦截，返回未认证错误	通过

测试结果表明，35 个功能测试用例均通过验证，系统在 Agent 管理、工作流执行、人工审核、知识库检索和用户认证等核心业务链路上能够满足预期功能要求。

系统性能测试

性能测试覆盖核心管理类接口的响应时间、SSE 实时推流的端到端延迟和工作流并发执行三类场景，测试工具为 Apache JMeter 5.x（接口压测）和 Chrome DevTools Network 面板（SSE 延迟测量）。

针对核心管理类接口（审批查询、审批恢复、执行详情、登录），使用 JMeter 在 50 并发用户、持续 60 秒的条件下进行压测，结果如表 5.4 所示。

API 响应时间测试结果

接口	P50 (ms)	P95 (ms)	P99 (ms)
GET /api/workflow/reviews/pending	18	45	78
GET	32	68	105

接口	P50 (ms)	P95 (ms)	P99 (ms)
/api/workflow/reviews/{id}			
POST	55	120	198
/api/workflow/reviews/resume			
POST	41	96	152
/api/user/login			
GET	22	52	89
/api/workflow/execution/{id}			

测试结果表明，核心管理类接口 P50 延迟均低于 60 ms，P99 延迟均低于 200 ms，满足第 2 章所确定的“核心管理接口 P95 低于 200 ms”的性能需求指标。

针对 SSE 实时推流，通过 Chrome DevTools Network 面板对 100 次执行样本进行端到端延迟测量，平均延迟 142 ms、P95 延迟 298 ms、最大延迟 387 ms，且 100 次样本中无 SSE 连接中断记录，验证了服务端 15 秒心跳保活机制的有效性，满足“SSE 推流 P95 低于 500 ms”的性能指标。

针对工作流并发执行，模拟同一 Agent 下 5 路并发工作流实例，各执行实例独立运行，节点状态、上下文与审批队列均无数据串扰，全部正常完成。测试结果表明，当前实现能够支撑基本并发执行场景，并在 IO 密集型节点调度（大模型推理与知识向量检索）中保持较好的响应稳定性。

系统部署说明

除功能和性能测试外，本文还对系统的部署方式进行说明。系统采用前后端分离和容器化部署方式，生产环境通过 Docker Compose 统一编排前端、后端和基础中间件。前端 React 应用构建后由 Nginx 容器提供静态资源访问，并将 /api 请求转发至后端服务；后端 Spring Boot 服务运行在 Java 21 环境中，通过容器网络访问 MySQL、Redis、MinIO 和 Milvus。该部署方式将应用服务与基础设施依赖统一纳入编排文件，降低了服务器初始化和环境迁移成本。

部署架构

生产部署使用 docker-compose.prod.yml 进行服务编排，主要服务组成如表 5.5 所示。

系统部署服务组成

服务	部署组件	作用说明
前端服务	Nginx + React 静态资源	提供系统页面访问入口，并代理后端接口请求
后端服务	Spring Boot + Java 21	承载认证、Agent 管理、工作流调度、人工审核和知识库接口
数据库	MySQL 8.0	存储用户、Agent、版本、执行实例、审核记录和知识库元数据
缓存队列	Redis 7	支撑验证码缓存、分布式锁、审批待办队列和运行状态缓存
对象存储	MinIO	存储知识库上传的原始文档和 Milvus 相关对象数据
向量数据库	Milvus 2.3 + etcd	存储文档向量索引，支持知识库语义检索

部署时，各容器加入同一 Docker 网络，后端通过服务名访问依赖组件。MySQL、MinIO、Milvus 和后端日志均使用 Docker 数据卷持久化，避免容器重启造成数据丢失。生产环境中仅前端访问端口、MinIO 管理端口和 Milvus 调试端口按需映射到宿主机，MySQL 与 Redis 默认仅在容器网络内访问，以减少数据库和缓存服务直接暴露带来的安全风险。

部署流程

系统部署前需要准备 Docker、Docker Compose、服务器端口和环境变量文件。部署时先复制 `.env.prod.example` 生成 `.env.prod`，并填写数据库密码、Redis 密码、MinIO 密码、JWT 密钥、邮件 SMTP 账号以及允许访问的前端域名。核心环境变量如表 5.6 所示。

生产环境关键配置项

配置项	说明
PRIMARY_DB_PASSWORD	MySQL 数据库访问密码
REDIS_PASSWORD	Redis 访问密码，用于缓存和分布

配置项	说明
	式锁连接
MINIO_ROOT_PASSWORD	MinIO 管理员密码，用于对象存储访问
JWT_SECRET	JWT 签名密钥，用于用户登录令牌签发与校验
MAIL_HOST / MAIL_USERNAME / MAIL_PASSWORD	邮件验证码发送所需 SMTP 配置
CORS_ALLOWED_ORIGINS	允许访问后端接口的前端域名

完成配置后，在服务器的 docker 目录下执行部署命令。系统支持使用已经构建好的镜像进行拉取部署，也支持在服务器上根据源码构建镜像后启动。典型部署命令如下。

```
cp .env.prod.example .env.prod
docker compose -f docker-compose.prod.yml pull
docker compose -f docker-compose.prod.yml up -d
docker compose -f docker-compose.prod.yml ps
```

服务启动后，前端容器对外提供页面访问入口，后端容器通过健康检查接口暴露运行状态。部署完成后可通过访问前端首页、调用后端健康检查接口、查看容器状态和检查关键日志来确认系统是否启动成功。

部署验证与运行维护

部署验证主要包括页面访问验证、接口健康检查、中间件连通性验证和日志检查四个方面。首先访问前端首页，确认 Nginx 能够正确返回静态页面；然后访问后端健康检查接口，确认 Spring Boot 服务已启动并能够连接数据库、Redis、MinIO 和 Milvus；最后查看后端日志和容器状态，确认初始化脚本、数据卷挂载和服务依赖均正常。

```
docker compose -f docker-compose.prod.yml ps
curl http://localhost:8090/actuator/health
docker compose -f docker-compose.prod.yml logs -f backend
```

运行维护方面，系统通过 Docker 数据卷持久化 MySQL、MinIO 和 Milvus 数据，常规停止命令不会清理数据卷。版本更新时，可先拉取新镜像，再执行 docker compose -f docker-compose.prod.yml up -d 触发服务重建；如果需要回滚，则通过指定镜像标签重新启动对应版本。由于系统包含人工审核记录、执行日志和知识库文档等重要数据，生产环境部署时还需要定期备份数据库和对象存储数据，并避免使用会删除数据卷的清理命令。

智能体编排系统总结与展望

研究结论

随着大语言模型能力持续提升，智能体（AI Agent）应用在企业流程自动化、内容生成与知识增强等场景中迅速普及。然而相对于 Python 生态中已较为成熟的智能体编排框架，Java 生态长期缺乏原生的可视化 Agent 编排平台，开发者难以在企业服务现有技术栈下快速构建符合业务规范的智能体应用。

针对上述问题，本文设计并实现了一套基于 Spring Boot 的 AI Agent 智能体编排系统，围绕 Agent 配置、流程执行、人工审核、知识增强与访问安全五个方面完成系统设计与实现，主要研究成果如下：

（1）实现了 Agent 管理与可视化编排能力。系统通过可视化画布支持节点拖拽、参数配置、边连接与流程保存，并对 Agent 生命周期提供创建、更新、发布、版本查询与回滚机制，使工作流定义具备可配置、可追溯、可复用的工程化特征。

（2）构建了事件驱动的 DAG 工作流调度执行机制。系统基于拓扑排序实现了“就绪节点驱动”的异步调度策略，支持无依赖节点的自动并行执行、条件分支与路径剪枝，并通过执行聚合根统一维护节点状态、上下文数据和执行推进逻辑，保证了工作流执行过程的可控性与可维护性。

（3）实现了人工检查点与工作流恢复机制。系统支持节点执行前与执行后两种审核触发阶段，审核人可查看执行上下文并对中间结果进行修改后放行，也可拒绝并终止流程。结合已审批节点跳过机制、分布式锁与版本号控制，保障了多人并发审批场景下的状态一致性与审计可追踪性。

（4）实现了知识库与检索增强链路。系统支持知识数据集管理、文档上传、异步解析、文本分块与向量化入库，并基于向量数据库提供语义检索能力。在工作流运行过程中，检索结果可作为上下文注入 LLM 节点，从而提升模型回答与业务知识之间的相关性。

（5）实现了用户认证与系统安全支撑能力。系统完成了邮箱注册、登录、验证码校验、密码找回等基础身份能力，并通过密码哈希加密、验证码发送频率限制、登录失败限流与令牌认证等机制，为 Agent 管理、知识库管理和工作流执行等功能提供统一的访问控制基础。

综合来看，本文实现的系统已打通“可视化编排 → 工作流执行 → 人工审核 → 知识增强 → 安全访问”的核心闭环，为 Java 生态下 AI Agent 系统的落地提供了可复用的实现思路。经测试与部署验证，系统通过 42 个单元测试用例（核心领域层行覆盖率 78%、分支覆盖率 71%）和 35 个功能测试用例的全面验证，主要管理类接口 P99 延迟低于 200

ms，SSE 实时推流 P95 延迟为 298 ms，并可通过 Docker Compose 完成前后端与基础中间件的容器化部署，具备企业级 MVP 演示与业务验证能力。

研究不足

受时间与精力限制，本文工作仍存在以下不足：

（1）Agent 编排能力仍以单工作流为主。当前系统已支持可视化编排与版本管理，但尚未支持复杂子流程复用、组合模板沉淀与跨 Agent 协同编排，复杂业务场景的编排表达能力有限。

（2）知识增强链路较为基础。当前文档分块采用固定策略，未根据文档结构与领域特征进行自适应；检索阶段未引入重排模型，且对命中片段的解释性支持不足。

（3）人工审核交互体验有待提升。当前审批页面以结构化文本和 JSON 形式展示节点输入输出，对非技术用户不够友好，缺乏富文本、表格、图片等多类型内容的差异化展示能力。

（4）性能与高可用验证范围有限。现阶段测试聚焦于核心链路可用性，尚未在更高并发规模、服务重启恢复与分布式部署等场景下进行充分验证。

未来工作展望

针对上述不足，后续可在以下方向继续研究和优化：

（1）增强 Agent 编排的复用与组合能力。引入子工作流、流程模板与并行聚合节点等机制，提升复杂业务场景下的编排表达力。

（2）完善知识检索增强效果。探索自适应分块策略、召回重排模型与知识来源可解释展示，提升知识命中质量与生成结果稳定性。

（3）优化人工审核交互与通知机制。结合字段级渲染、差异对比视图与邮件、即时消息通知机制，提升审核节点在企业协作场景中的可用性与响应效率。

（4）加强系统性能与高可用能力。围绕更大规模并发调度、SSE 长连接稳定性、执行断点恢复与分布式部署进行工程优化，进一步提升系统在生产环境中的稳定性。

（5）拓展多智能体协作与工具生态。在现有单 Agent 编排能力基础上，探索多智能体协同机制、更丰富的工具接入方式以及与 Spring AI 新特性的深度整合，扩展系统的适用范围与协作能力。

时光荏苒，大学四年的学习生活即将画上句号。在本篇毕业论文完成之际，我谨向在学习、科研、项目开发和论文写作过程中给予我指导、帮助与支持的老师、同学、朋友和家人致以诚挚的感谢。

首先，衷心感谢我的指导老师在设计与论文写作全过程中给予的悉心指导。从课题选题、需求分析、系统设计，到功能实现、论文结构调整和格式规范，老师都提出了许多具体而中肯的意见，使我能够在较为复杂的工程实践中保持清晰的研究目标和技术主线。在论文反复修改过程中，老师严谨认真的治学态度、耐心细致的指导方式和对工程细节的严格要求，使我受益匪浅，也让我更加深刻地认识到理论分析、系统设计与工程实现相结合的重要性。

感谢西南科技大学计算机科学与技术学院各位老师四年来的教导与培养。大学期间所学习的软件工程、数据库系统、Java Web 开发、人工智能、计算机网络等课程，为本课题的完成奠定了坚实基础。正是在系统化的专业学习和实践训练中，我逐步掌握了软件系统分析、架构设计、接口开发、数据建模和测试验证等能力，这些知识和能力在本文所实现的 AI Agent 智能体编排系统中得到了综合运用。

感谢同学和朋友们在项目开发、系统测试和论文撰写期间给予的帮助与鼓励。在系统功能联调、界面体验优化、测试用例设计和论文表述修改过程中，大家提出的建议帮助我发现并改进了许多容易忽视的问题，使系统功能更加完整，论文表达也更加清晰。与同学们共同学习、讨论问题和解决困难的经历，是我本科阶段十分珍贵的记忆。

感谢开源社区及相关技术生态提供的支持。Spring Boot、React、TypeScript、React Flow、Milvus、Redis、MySQL 等开源技术为系统的设计与实现提供了重要基础，也使我能够在真实工程环境中理解现代软件开发的技术体系和协作方式。通过查阅文档、阅读源码和解决实践问题，我进一步提升了自主学习能力和工程问题分析能力。

最后，感谢我的家人在求学期间给予的理解、支持与鼓励。正是他们在生活和精神上的陪伴，使我能够在设计、开发和论文写作过程中保持耐心与信心。毕业论文的完成既是本科阶段学习成果的一次总结，也是未来继续学习和成长的新起点。今后我将继续保持严谨求实的态度，把本科阶段积累的专业知识和工程实践能力运用到后续学习与工作中。

参考文献

Evans, Eric. 2016. *领域驱动设计：软件核心复杂性应对之道*. Translated by 赵俐 and 盛海艳. 人民邮电出版社.

Lewis, Patrick, Ethan Perez, Aleksandra Piktus, et al. 2020. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *Advances in Neural Information Processing Systems* 33: 9459–74.

<https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>.

Wang, Jianguo, Xiaomeng Yi, Rentong Guo, et al. 2021. "Milvus: A Purpose-Built Vector Data Management System." *Proceedings of the 2021 International Conference on Management of Data*, 2614–27. <https://doi.org/10.1145/3448016.3457550>.

Wang, Lei, Chen Ma, Xueyang Feng, et al. 2024. "A Survey on Large Language Model Based Autonomous Agents." *Frontiers of Computer Science* 18 (6): 186345. <https://doi.org/10.1007/s11704-024-40231-1>.

Wu, Qingyun, Gagan Bansal, Jieyu Zhang, et al. 2023. "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation." *arXiv Preprint arXiv:2308.08155*. <https://arxiv.org/abs/2308.08155>.

Xi, Zhiheng, Wenxiang Chen, Xin Guo, et al. 2023. "The Rise and Potential of Large Language Model Based Agents: A Survey." *arXiv Preprint arXiv:2309.07864*, ahead of print. <https://doi.org/10.48550/arXiv.2309.07864>.

Yao, Shunyu, Jeffrey Zhao, Dian Yu, et al. 2023. "ReAct: Synergizing Reasoning and Acting in Language Models." *International Conference on Learning Representations*. https://openreview.net/forum?id=WE_vluYUL-X.

刘雪颖, 云静, 李博, 史晓国, and 张钰莹. 2025. "基于大型语言模型的检索增强生成综述." *计算机工程与应用* 61 (13): 1–25. http://kns--cnki--net--https.cnki.mdjsf.utuvpn.utuedu.com:9000/kcms2/article/abstract?v=ZMT4ddBD2nwMYTWQL6F0Y8pzzHCLtyxGjAp1tuvyxPF-S-CZ4cbsrWVoLAg46bIjzdwPThfCAh4FKK65dfqdSxu46OGnni5Y9qBVQx68jemoiMP4MGkl8yB7u_KRvoKxGr4PnT-ILZUg6pwowDcQ3az6THDm6kw1Y8lkK9DTPoRcgYy_SdmVA==&uniplatform=NZKP T&language=CHS.

包晓安, 曹云棣, and 张娜. 2019. "基于格分布方差的多目标云 workflow 调度算法." *电信科学* 35 (2): 1–13. <https://doi.org/10.11959/j.issn.1000-0801.2019035>.

周志明. 2019. *深入理解Java 虚拟机: JVM 高级特性与最佳实践*. 机械工业出版社.

奚志恒, 陈文翔, 郭昕, et al. 2026. "基于大语言模型的智能体: 发展与未来展望." *中国科学: 信息科学* 56 (2): 485–86. <https://www.sciengine.com/doi/pdfView/9E6E506A38D04A1E912191CDA9B13C55>.

尹建业, and 苏笑扬. 2025. "基于 Dify 工作流的测试项批量生成工具设计与实现." *中国战略新兴产业*, no. 29: 159–61. <http://kns--cnki--net-->

徐健锐, and 朱会娟. 2019. “云计算环境中面向 DAG 任务的多目标调度算法.” *计算机应用研究* 36 (1): 31–36. <https://doi.org/10.19734/j.issn.1001-3695.2017.07.0683>.

李锦哲, 赵辉, 支文韬, and 王静. 2026. “混合多云环境下多目标优化工作流调度方法.” *西安电子科技大学学报*, ahead of print. <https://doi.org/10.19665/j.issn1001-2400.20260307>.

袁友伟, 鲍泽前, 俞东进, and 李万清. 2018. “云环境下基于多目标的多科学工作流调度算法.” *软件学报* 29 (11): 3326–39. <https://jos.org.cn/html/2018/11/5477.htm>.

郭先会, 张梦姣, and 马军. 2024. “基于大语言模型的智能体构建研究综述.” *通信技术* 57 (9). <https://doi.org/10.3969/j.issn.1002-0802.2024.09.001>.

陈旭. 2024. “基于大语言模型的自主智能体概述.” *Proceedings of the 23rd Chinese National Conference on Computational Linguistics*, 141–50.
<https://aclanthology.org/2024.ccl-2.8/>.

黄健宏. 2014. *Redis 设计与实现*. 机械工业出版社.